

Everyone loves lists



You're asking one question: "What makes Python different?"

The short answer is: *lots of things*. The longer answer starts by stating that there's lots that's familiar, too. Python is a lot like any other *general-purpose* programming language, with **statements**, **expressions**, **operators**, **functions**, **modules**, **methods**, and **classes**. All the *usual stuff*, really. And then there's the other stuff Python provides that makes the programmer's life—*your* life—that little bit easier. You'll start your tour of Python by learning about **lists**. But, before getting to that, there's another important question that needs answering...

What's to like about Python?

Lots. Rather than tell you, this book's goal is to *show* you the greatness that is Python.



Before diving head first into Python, let's get a bit of housekeeping out of the way.

To work with and execute the Python code in this book, you need a copy of the **Python 3 interpreter** on your computer. Like a lot of things to do with Python, it's not difficult to install the interpreter. Assuming, of course, it's not already there...

Install Python 3

Before you *write* and *run* Python code, you need to make sure the Python interpreter is *on* your computer. In this book, you'll start out with Release 3 of Python, the very latest (and best) version of the language.

A release of Python might already be on your computer. Mac OS X comes with Python 2 preinstalled, as do most versions of Linux (which can also ship with Release 3). Windows, in contrast, doesn't include any release of Python. Let's check your computer for Python 3. Open up a command-line prompt and, if you are using Mac OS X or Linux, type:

```
python3 -V
```

On Windows, use this command:

```
c:\Python31\python.exe -V
```

That's an UPPERCASE
"V", by the way.



If Python 3 is missing from your computer, download a copy for your favorite OS from the www.python.org website.

Using the UPPERCASE "V" results in the Python version appearing on screen.

Without the UPPERCASE "V", you are taken into the Python interpreter.

Use the quit() command to exit the interpreter and return to your OS prompt.

```
File Edit Window Help WhichPython?
$ python3 -V
Python 3.1.2
$
$ python3
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "help", "copyright", "credits" or "license" for more info.
>>>
>>> quit()
$
```

When you install Python 3, you also get **IDLE**, Python's simple—yet surprisingly *useful*—integrated development environment. IDLE includes a color syntax-highlighting editor; a debugger, the Python Shell, and a complete copy of Python 3's online documentation set.

Let's take a quick look at IDLE.

Use IDLE to help learn Python

IDLE lets you *write* code in its full-featured code editor as well as *experiment* with code at the **Python Shell**. You'll use the code editor later in this book but, when learning Python, IDLE's shell really rocks, because it lets you try out new Python code *as you go*.

When you first start IDLE, you are presented with the “triple chevron” prompt (`>>>`) at which you enter code. The shell takes your code statement and *immediately* executes it for you, displaying any results produced on screen.

IDLE knows all about Python syntax and offers “completion hints” that pop up when you use a **built-in function** like `print()`. Python programmers generally refer to built-in functions as **BIFs**. The `print()` BIF displays messages to *standard output* (usually the screen).

Unlike other C-based languages, which use `{` and `}` to delimit blocks, Python uses indentation instead.

Enter your code at the `>>>` prompt.

See results immediately.

```
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>>
>>> print("You can experiment with code within IDLE's shell. Cool, eh?")
You can experiment with code within IDLE's shell. Cool, eh?
>>> if 43 > 42:
    print("Don't panic!")
Don't panic!
>>> |
```

IDLE uses colored syntax to highlight your code. By default, built-in functions are purple, strings are green, and language keywords (like `if`) are orange. Any results produced are in blue. If you hate these color choices, don't worry; you can easily change them by adjusting IDLE's preferences.

IDLE also knows all about Python's indentation syntax, which requires code blocks be **indented**. When you start with Python, this can be hard to get used to, but IDLE keeps you straight by automatically indenting as needed.

**IDLE knows
Python's syntax
and helps you
conform to
the Python
indentation rules.**

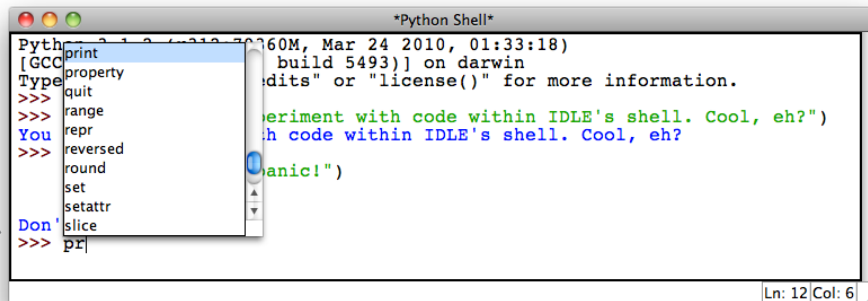
Work effectively with IDLE

IDLE has lots of features, but you need to know about only a few of them to get going.

TAB completion

Start to type in some code, and then press the **TAB** key. IDLE will offer suggestions to help you complete your statement.

Type "pr" and then TAB at the >>> prompt to see IDLE's list of command completion suggestions.



This is how IDLE looks on my computer. It might look a little different on yours, but not by much. (And, yes, it's meant to look this ugly.)

Recall code statements

Press **Alt-P** to recall the *previous* code statement entered into IDLE or press **Alt-N** to move to the *next* code statement (assuming there is one). Both key combinations can be used to cycle rapidly through all of the code you've entered into IDLE, re-executing any code statements as needed.

Alt-P for Previous

Alt-N for Next

Unless you're on a Mac, in which case it's Ctrl-P and Ctrl-N.

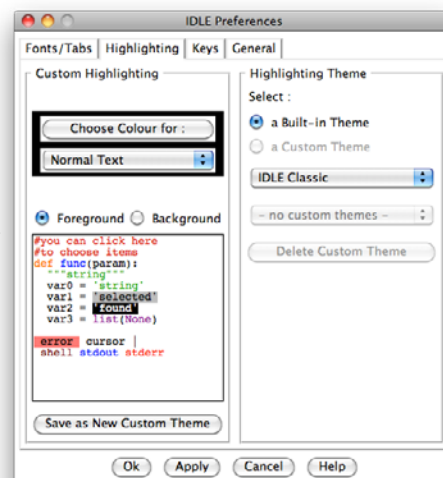
Edit recalled code

Once you recall your code statement, you can *edit* it and move around the statement using the arrow keys. It's possible to edit any statement that you've previously entered, even code statements that span multiple lines.

Adjust IDLE's preferences

IDLE's preferences dialog lets you adjust its default behavior to your tastes. There are four tabs of settings to tweak. You can control font and tab behavior, the colors used to syntax highlight, the behavior of certain key-combinations, and IDLE's start-up settings. So, if shocking pink strings is really your thing, IDLE gives you the power to change how your code looks on screen.

Tweak IDLE to your heart's content.

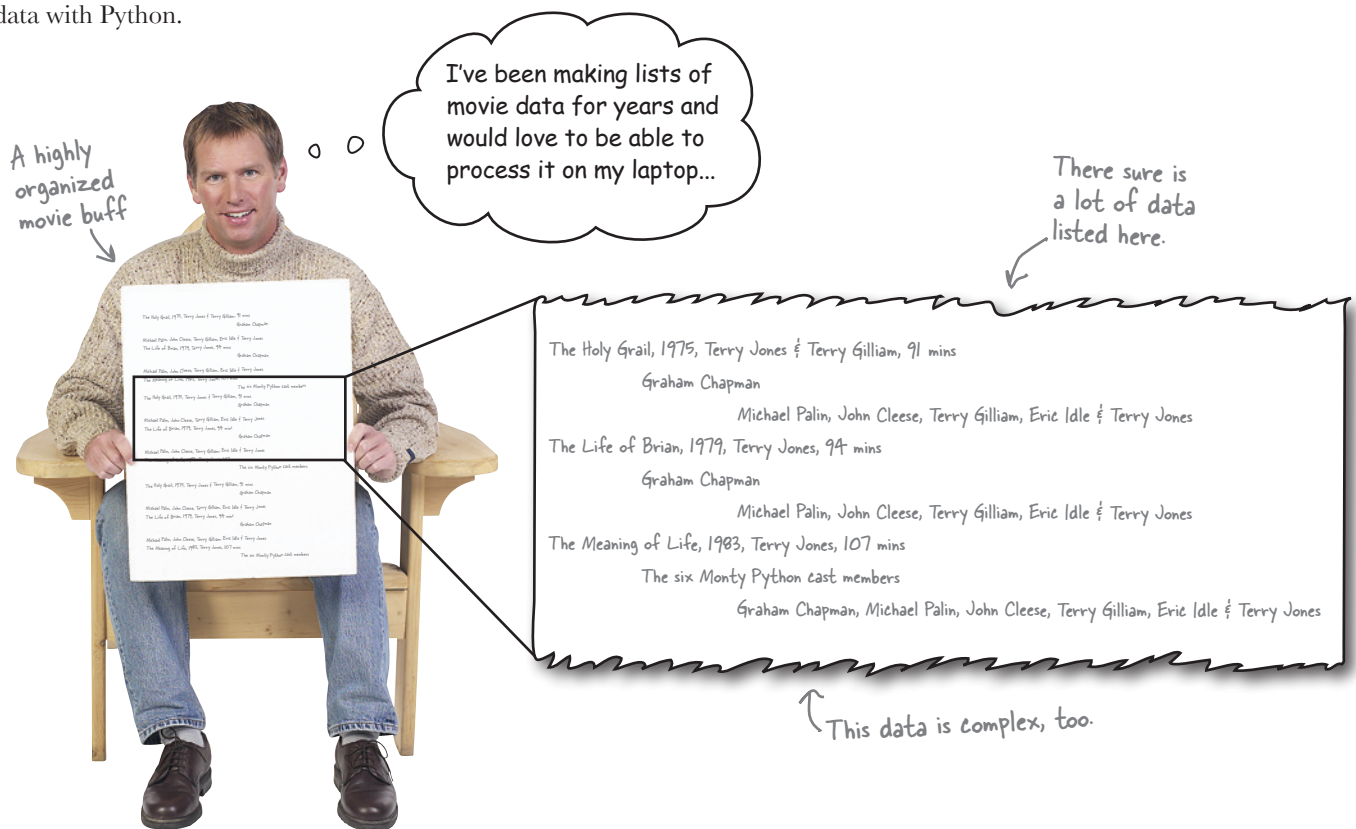


Deal with complex data

Any program of any worth that you create has to work with **data**. Sometimes, the data is simple and straightforward—*easy to work with*. Other times, the data you have to work with is *complex* in its structure and meaning, forcing you to work hard to make sense of it all, let alone write code to process it.

To tame complexity, you can often arrange your data as a **list**: there's the list of customers, your friend's list, the shopping list, and your to-do list (to name a few). Arranging data in lists is so common that Python makes it easy for you to create and process lists in code.

Let's look at some complex data before learning how to create and process list data with Python.



On first glance, this collection of data does indeed look quite complex. However, the data appears to conform to some sort of structure: there's a line for a list of basic movie facts, then another line for the lead actor(s), followed by a third line listing the movie's supporting actors.

This looks like a structure you can work with...

Create simple Python lists

Let's start with the following simple list of movie titles and work up from there:

The Holy Grail

The Life of Brian

The Meaning of Life

A short list of some
Monty Python movies

Here's the same list written in a way that Python understands:

```
movies = ["The Holy Grail",  
          "The Life of Brian",  
          "The Meaning of Life"]
```

To turn the human-friendly list into a Python-friendly one, follow this four-step process:

- 1 **Convert** each of the names into strings by surrounding the data with quotes.
- 2 **Separate** each of the list items from the next with a comma.
- 3 **Surround** the list of items with opening and closing square brackets.
- 4 **Assign** the list to an identifier (`movies` in the preceding code) using the assignment operator (`=`).

It's perfectly OK to put your list creation code *all on one line*, assuming, of course, that you have room:

```
movies = ["The Holy Grail", "The Life of Brian", "The Meaning of Life"]
```

↑
This works, too.



Hang on a second! Aren't you forgetting something? Don't you need to declare type information for your list?

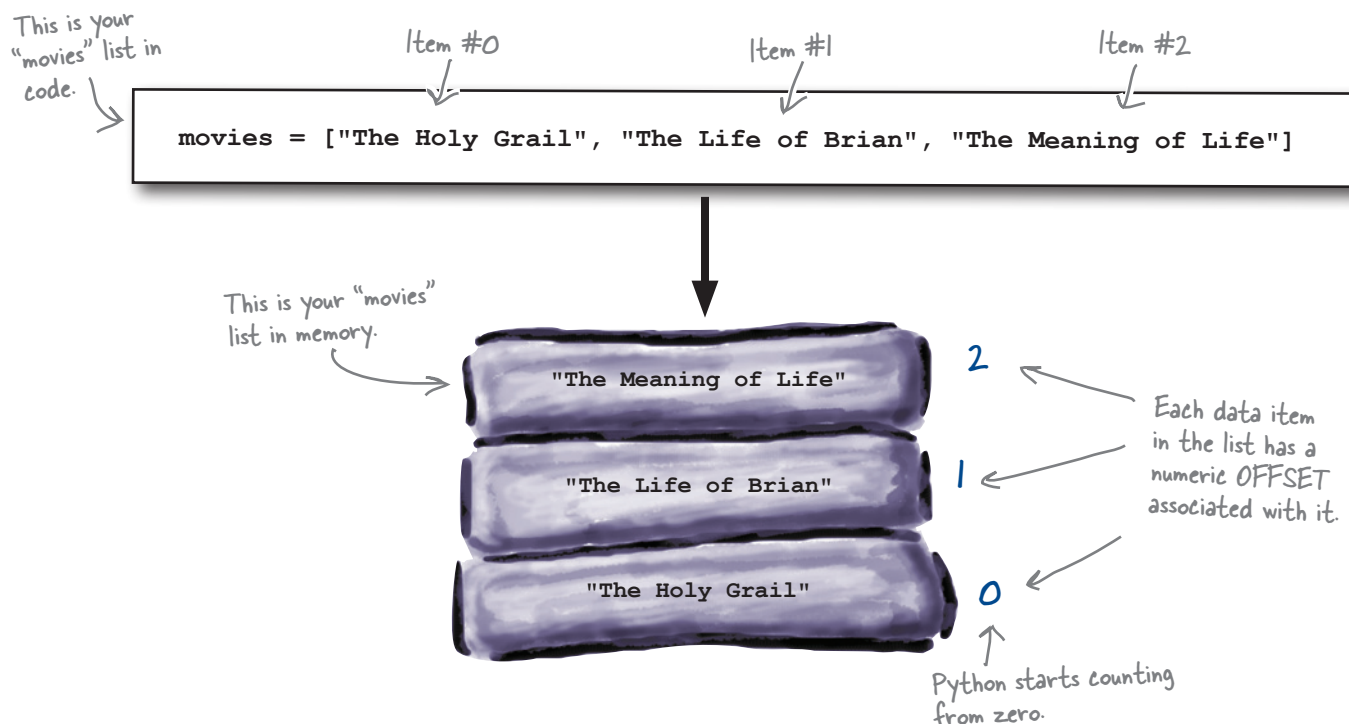
No, because Python's variable identifiers don't have a type.

Many other programming languages insist that every identifier used in code has type information declared for it. Not so with Python: identifiers are simply names that refer to a data object *of some type*.

Think of Python's list as a **high-level collection**. The type of the data items is not important to the list. It's OK to state that your `movies` list is a "collection of strings," but Python *doesn't* need to be told this. All Python needs to know is that you need a list, you've given it a name, and the list has some data items in it.

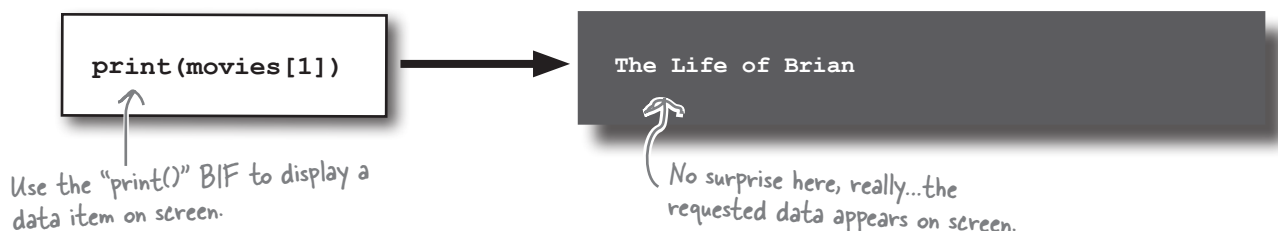
Lists are like arrays

When you create a list in Python, the interpreter creates an array-like data structure in memory to hold your data, with your data items stacked from the bottom up. Like array technology in other programming languages, the first slot in the stack is numbered 0, the second is numbered 1, the third is numbered 2, and so on:



Access list data using the square bracket notation

As with arrays, you can access the data item in a list slot using the *standard square bracket offset notation*:



Let's use IDLE to learn a bit about how lists work.



An IDLE Session

Lists in Python might look like arrays, but they are much more than that: they are full-blown Python *collection objects*. This means that lists come with ready-to-use functionality in the form of **list methods**.

Let's get to know some of Python's list methods. Open up IDLE and follow along with the code entered at the `>>>` prompt. You should see exactly the same output as shown here.

Start by defining a list of names, which you then display on screen using the `print()` BIF. Then, use the `len()` BIF to work out how many data items are in the list, before accessing and displaying the value of the second data item:

```
>>> cast = ["Cleese", 'Palin', 'Jones', "Idle"]
>>> print(cast)
['Cleese', 'Palin', 'Jones', 'Idle']
>>> print(len(cast))
4
>>> print(cast[1])
Palin
```

It's OK to invoke a BIF on the results of another BIF.

With your list created, you can use list methods to add a single data item to the end of your list (using the `append()` method), remove data from the end of your list (with the `pop()` method), and add a collection of data items to the end of your list (thanks to the `extend()` method):

```
>>> cast.append("Gilliam")
>>> print(cast)
['Cleese', 'Palin', 'Jones', 'Idle', 'Gilliam']
>>> cast.pop()
'Gilliam'
>>> print(cast)
['Cleese', 'Palin', 'Jones', 'Idle']
>>> cast.extend(["Gilliam", "Chapman"])
>>> print(cast)
['Cleese', 'Palin', 'Jones', 'Idle', 'Gilliam', 'Chapman']
```

Methods are invoked using the common "." dot notation.

It's another list: items separated by commas, surrounded by square brackets.

Finally, find and remove a *specific* data item from your list (with the `remove()` method) and then add a data item before a *specific* slot location (using the `insert()` method):

```
>>> cast.remove("Chapman")
>>> print(cast)
['Cleese', 'Palin', 'Jones', 'Idle', 'Gilliam']
>>> cast.insert(0, "Chapman")
>>> print(cast)
['Chapman', 'Cleese', 'Palin', 'Jones', 'Idle', 'Gilliam']
```

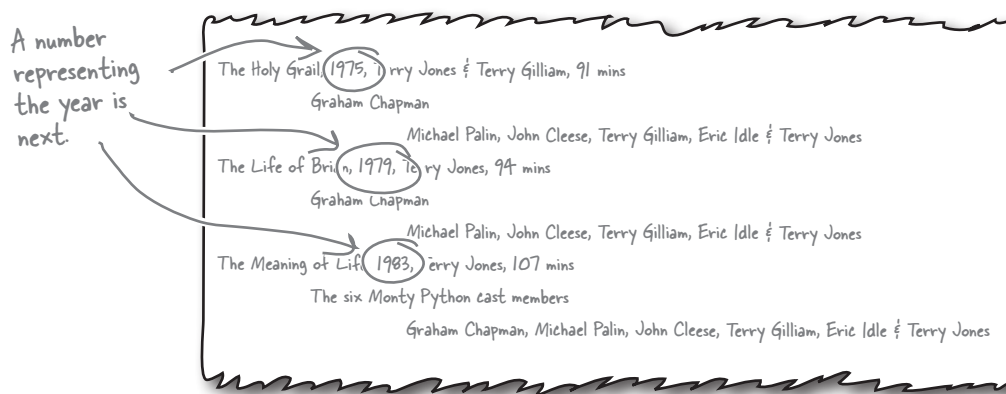
After all that, we end up with the cast of Monty Python's Flying Circus!

Add more data to your list

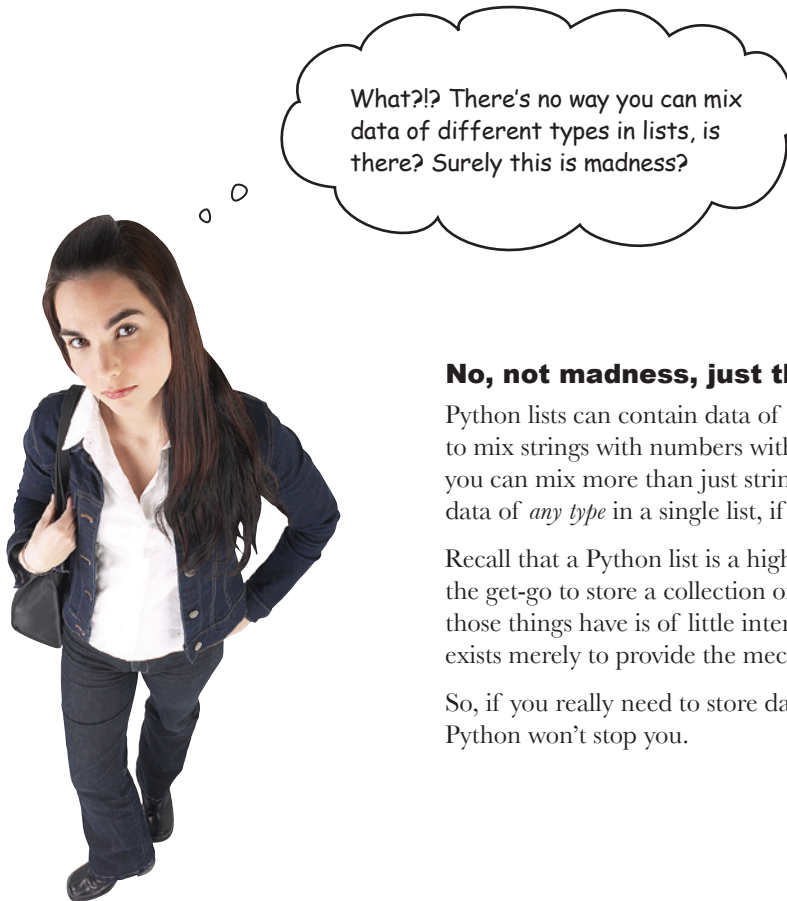
With your list of movie names created, now you need to add more of the movie buff's complex data to it. You have a choice here:



Either strategy works. Which works best for *you* depends on what you are trying to do. Let's recall what the movie buff's data looks like:



The next piece of data you need to add to your list is a **number** (which represents the year the movie was released), and it must be inserted *after* each movie name. Let's do that and see what happens.



No, not madness, just the way Python works.

Python lists can contain data of **mixed type**. It's perfectly OK to mix strings with numbers within the *same* Python list. In fact, you can mix more than just strings and numbers; you can store data of *any type* in a single list, if you like.

Recall that a Python list is a high-level collection, designed from the get-go to store a collection of “related things.” What *type* those things have is of little interest to the list, because the list exists merely to provide the mechanism to store data in list form.

So, if you really need to store data of mixed type in a list, Python won't stop you.



Exercise

Let's take a bit of time to try to work out which strategy to use when adding data to your list *in this case*.

Given the following list-creation code:

```
movies = ["The Holy Grail", "The Life of Brian", "The Meaning of Life"]
```

- 1 Work out the Python code required to insert the numeric year data into the preceding list, changing the list so that it ends up looking like this:

```
["The Holy Grail", 1975, "The Life of Brian", 1979, "The Meaning of Life", 1983]
```

Write your
insertion
code here. →

.....

.....

.....

.....

.....

.....

.....

- 2 Now write the Python code required to re-create the list with the data you need all in one go:

Write your
re-creation
code here. →

.....

.....

.....

.....

In this case, which of these two methods do you think is best? (Circle your choice).

1

or

2



Exercise Solution

Let's take a bit of time to try and work out which strategy to use when adding data to your list *in this case*.

Given the following list-creation code:

```
movies = ["The Holy Grail", "The Life of Brian", "The Meaning of Life"]
```

- 1 You were to work out the Python code required to insert the numeric year data into the preceding list:

Insert the first year
BEFORE the second list
item.

```
movies.insert(1, 1975)
```

Insert the second year
BEFORE the fourth list item.

```
movies.insert(3, 1979)
```

Did you get the math right?
After the first insertion, the
list grows, so you have to take
that into consideration when
working out where to do the
second insert.

```
movies.append(1983)
```

Then append the last year to
the end of the list.

- 2 You were also to write the Python code required to recreate the list with the data you need all in one go:

```
movies = ["The Holy Grail", 1975,
```

```
        "The Life of Brian", 1979,
```

```
        "The Meaning of Life", 1983]
```

Assign all your data to the "movies"
identifier. What was previously there is
replaced.

In this case, which of these two methods do you think is best? (You were to circle your choice.)

1

or

2

Yes, method 2 seems the better
option here...that is, for a small
list like this. Also, there's no
tricky counting to do.

Work with your list data

You often need to *iterate* over your list and perform some action on each item as you go along. Of course, it is always possible to do something like this, which works but *does not scale*:

Define a list and populate its items with the names of two movies.

```
fav_movies = ["The Holy Grail", "The Life of Brian"]

print(fav_movies[0])
print(fav_movies[1])
```

Display the value of each individual list item on the screen.

This is the list-processing code.

This code works as expected, making the data from the list appear on screen. However, if the code is later amended to add *another* favorite movie to the list, the list-processing code stops working as expected, *because the list-processing code does not mention the third item*.

Big deal: all you need to do is add another `print()` statement, right?

Yes, adding one extra `print()` statement works for one extra movie, but what if you need to add another hundred favorite movies? The *scale* of the problem defeats you, because adding all those extra `print()` statements becomes such a chore that you would rather find an excuse not to have to do.

It's time to iterate

Processing every list item is such a common requirement that Python makes it especially convenient, with the built-in **for** loop. Consider this code, which is a rewrite of the previous code to use a **for** loop:

Define a list and populate it just as you did before.

Use "for" to iterate over the list, displaying the value of each individual item on screen as you go.

```
fav_movies = ["The Holy Grail", "The Life of Brian"]

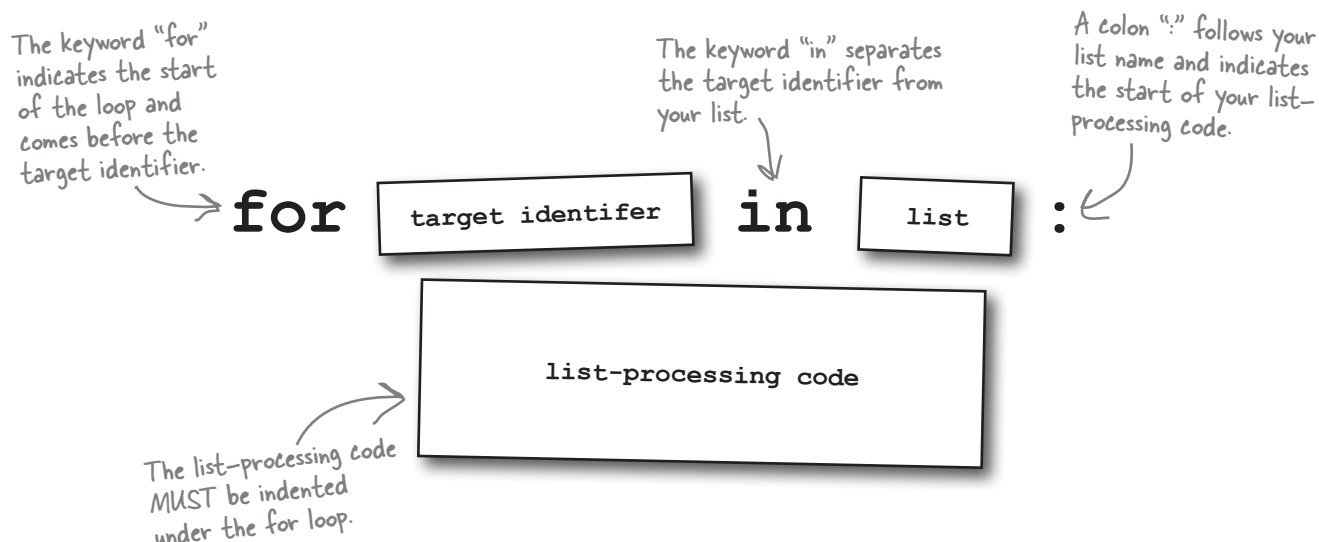
for each_flick in fav_movies:
    print(each_flick)
```

This is the list-processing code, using a for loop.

Using a for loop scales and works with any size list.

For loops work with lists of any size

Python's **for** loop exists to process lists and other *iterations* in Python. Lists are the most common iterated data structure in Python, and when you need to iterate a list, it's best to use **for**:



The *list-processing code* is referred to by Python programmers as the **suite**.

The **target identifier** is like any other name in your code. As your list is iterated over, the target identifier is assigned *each* of the data values in your list, in turn. This means that each time the loop code executes, the target identifier refers to a *different* data value. The loop keeps iterating until it exhausts all of your list's data, no matter how big or small your list is.

An alternative to using **for** is to code the iteration with a **while** loop.

Consider these two snippets of Python code, which perform the *same action*:

When you use "while", you have to worry about "state information," which requires you to employ a counting identifier.

```
count = 0
while count < len(movies):
    print(movies[count])
    count = count+1
```

```
for each_item in movies:
    print(each_item)
```

When you use "for", the Python interpreter worries about the "state information" for you.

These while and for statements do the same thing.

there are no Dumb Questions

Q: So...when iterating over a list, I should always use `for` instead of `while`?

A: Yes, unless you have a really good reason to use (or need the extra control of) a `while` loop. The `for` loop takes care of working from the start of your list and continuing to the end. It's next to impossible to get stung by an off-by-one error when you use `for`. This is not the case with `while`.

Q: So, lists aren't really like arrays then, because they do so much more?

A: Well...they are in that you can access individual data items in your list with the standard square bracket notation, but—as you've seen—Python's lists can do so much more. At Head First Labs, we like to think of lists as “arrays on steroids.”

Q: And they work this way only in Python 3, right?

A: No. There are certain enhancements to lists that were added in Python 3, but release 2 of Python has lists, too. All of what you've learned about lists so far will work with lists in Releases 2 and 3 of Python.

Q: Why are we using Python 3? What's wrong with Python 2, anyway? Lots of programmers seem to be using it.

A: Lots of programmers are using Python 2, but the future of Python development lies with Release 3. Of course, moving the entire Python community to Python 3 won't happen overnight, so there's an awful lot of projects that will continue to run on Release 2 for the foreseeable future. Despite 2's dominance *at the moment*, at Head First Labs we think the new bits in 3 are well worth the added investment in learning about them now. Don't worry: if you know 2, Python 3 is easy.

Q: Seeing as Python's lists shrink and grow as needed, they must not support bounds-checking, right?

A: Well, lists are *dynamic*, in that they shrink and grow, but they are *not magic*, in that they cannot access a data item that does not exist. If you try to access a nonexistent data item, Python responds with an `IndexError`, which means “out of bounds.”

Q: What's with all the strange references to Monty Python?

A: Ah, you spotted that, eh? It turns out that the creator of Python, *Guido van Rossum*, was reading the scripts of the Monty Python TV shows while designing his new programming language. When Guido needed a name for his new language, he chose “Python” as a bit of a joke (or so the legend goes).

Q: Do I need to know Monty Python in order to understand the examples?

A: No, but as they say in the official Python documentation: “it helps if you do.” But don't worry: you'll survive, even if you've never heard of *Monty Python*.

Q: I notice that some of your strings are surrounded with double quotes and others with single quotes. What's the difference?

A: There isn't any. Python lets you use either to create a string. The only rule is that if you start a string with one of the quotes, then you have to end it with the same quote; you can't mix'n'match. As you may have seen, IDLE uses single quotes when displaying strings within the shell.

Q: What if I need to embed a double quote in a string?

A: You have two choices: either escape the double quote like this: `\"`, or surround your string with single quotes.

Q: Can I use any characters to name my identifiers?

A: No. Like most other programming languages, Python has some rules that must be adhered to when creating names. Names can start with a letter character or an underscore, then include any number of letter characters, numbers, and/or underscores in the rest of the name. Strange characters (such as `%$£`) are not allowed and you'll obviously want to use names that have meaning *within the context of your code*. Names like `members`, `the_time`, and `people` are much better than `m`, `t`, and `p`, aren't they?

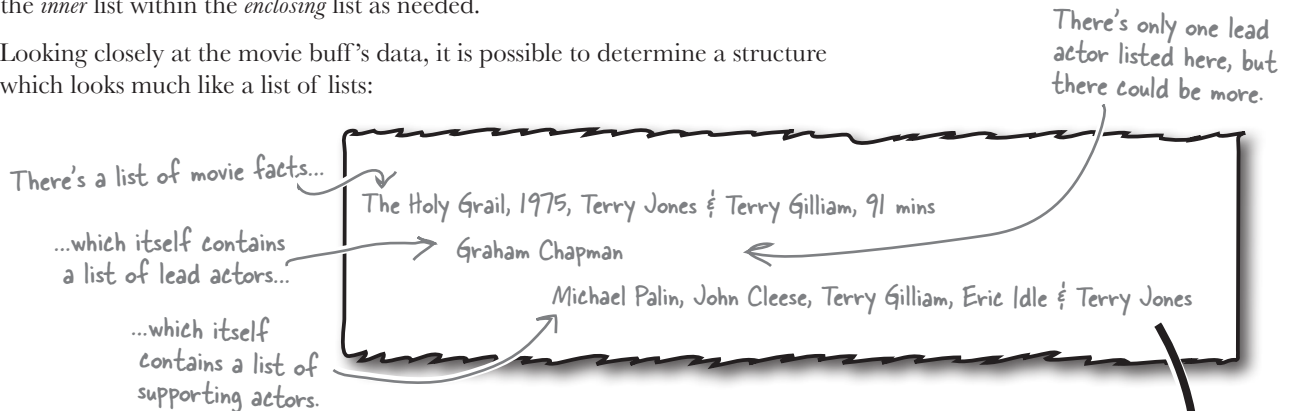
Q: Yes, good naming practice is always important. But what about case sensitivity?

A: Yes, Python is the “sensitive type,” in that Python code *is* case sensitive. This means that `msg` and `MSG` are two different names, so be careful. Python (and IDLE) will help with the problems that can occur as a result of this. For instance, you can use an identifier in your code only if it has been given a value; unassigned identifiers cause a runtime error. This means that if you type `msg` when you meant `msg`, you'll find out pretty quickly when Python complains about your code having a `NameError`.

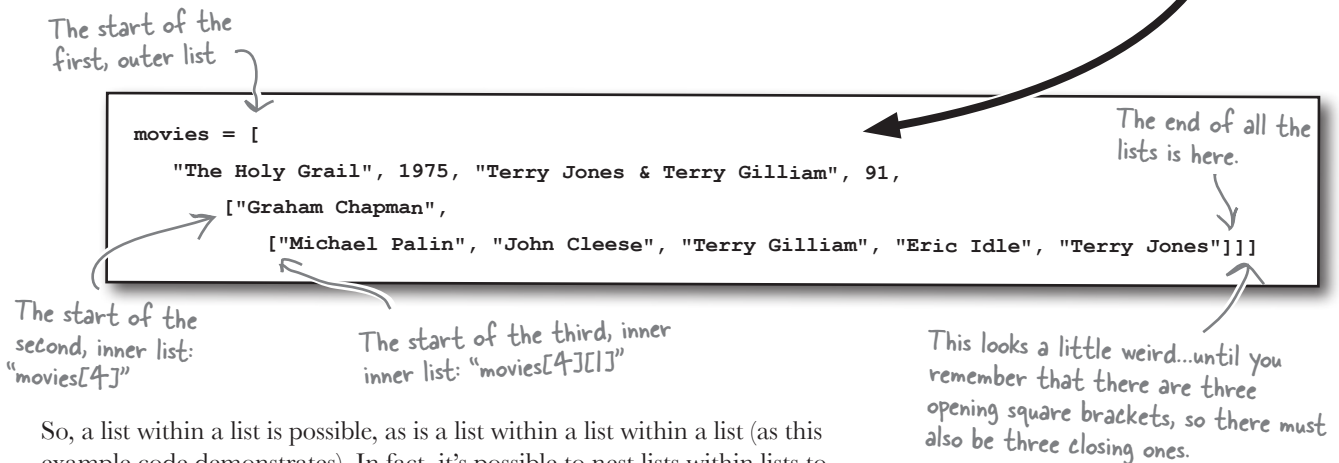
Store lists within lists

As you've seen, lists can hold data of mixed type. But it gets even better than that: lists can hold collections of anything, *including other lists*. Simply **embed** the *inner* list within the *enclosing* list as needed.

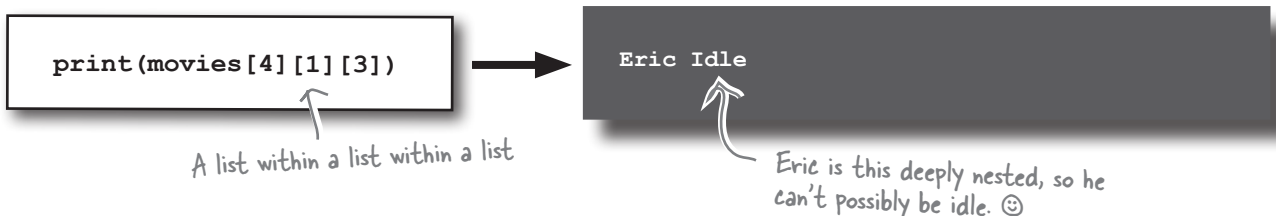
Looking closely at the movie buff's data, it is possible to determine a structure which looks much like a list of lists:



In Python, you can turn this real list of data into code with little or no effort. All you need to remember is that every list is a collection of items separated from each other with commas and surrounded with square brackets. And, of course, any list item can itself be another list:



So, a list within a list is possible, as is a list within a list within a list (as this example code demonstrates). In fact, it's possible to nest lists within lists to most any level with Python. And you can *manipulate* every list with its own list methods and *access* it with the square bracket notation:





An IDLE Session

Creating a list that contains another list is straightforward. But what happens when you try to process a list that contains another list (or lists) using the **for** loop from earlier in this chapter?

Let's use IDLE to work out what happens. Begin by creating the list of the movie data for "The Holy Grail" in memory, display it on screen, and then process the list with your **for** loop:

```
>>> movies = ["The Holy Grail", 1975, "Terry Jones & Terry Gilliam", 91,
               ["Graham Chapman", ["Michael Palin", "John Cleese",
                                     "Terry Gilliam", "Eric Idle", "Terry Jones"]]]

>>> print(movies)
['The Holy Grail', 1975, 'Terry Jones & Terry Gilliam', 91, ['Graham Chapman', ['Michael Palin',
'John Cleese', 'Terry Gilliam', 'Eric Idle', 'Terry Jones']]]

>>> for each_item in movies:
    print(each_item)
```

The Holy Grail
1975
Terry Jones & Terry Gilliam
91
['Graham Chapman', ['Michael Palin', 'John Cleese', 'Terry Gilliam', 'Eric Idle', 'Terry Jones']]

The list within a list within a list has been created in memory.

The "for" loop prints each item of the outer loop *ONLY*.

The inner list within the inner list is printed "as-is."

o o Your for loop is working OK. I think the trouble is that you haven't told it what to do with any inner lists that it finds, so it just prints everything, right?

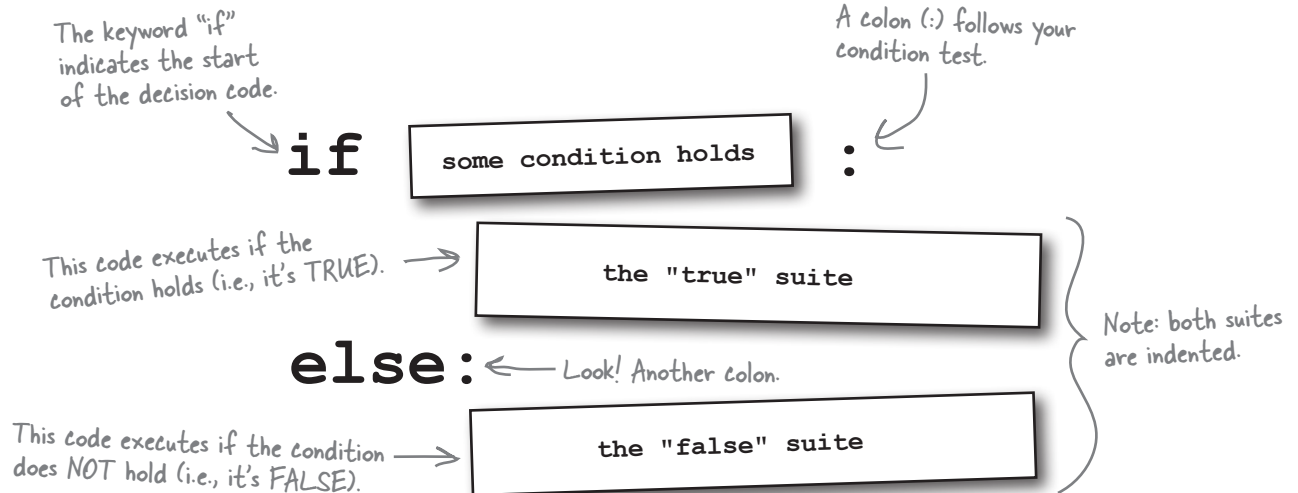
Yes, that's correct: the loop code isn't complete.

At the moment, the code within the loop simply prints each list item, and when it finds a list at a slot, it simply displays the entire list on screen. After all, the inner list is just another list item as far as the outer enclosing list is concerned. What's we need here is some mechanism to spot that an item in a list is in fact *another* list and take the appropriate action.

That sounds a little tricky. But can Python help?

Check a list for a list

Each time you process an item in your list, you need to check to see if the item is another list. If the item is a list, you need to process the **nested list** *before* processing the next item in your outer list. Deciding *what to do when* in Python follows the familiar **if... else...** pattern:



No surprises here, as the **if** statement in Python works pretty much as expected. But what *condition* do you need to check? You need a way to determine if the item currently being processed is a list. Luckily, Python ships with a BIF that can help here: `isinstance()`.

What's cool about the `isinstance()` BIF is that it lets you check if a *specific* identifier holds data of a *specific* type:

An IDLE Session

Let's use the IDLE shell to learn a little about how `isinstance()` works:

Create a short list and assign it to an identifier.

Ask if "names" is a list (it is).

Assign a number to an identifier.

Ask if "num_names" is a list (it isn't).

```

>>> names = ['Michael', 'Terry']

>>> isinstance(names, list)
True

>>> num_names = len(names)

>>> isinstance(num_names, list)
False
          
```

Refer to a Python type here. In this case, the type is "list".



Exercise

Here's a copy of the current list-processing code. Your task is to rewrite this code using an `if` statement and the `isinstance()` BIF to process a list that displays another list.

```
for each_item in movies:
    print(each_item)
```

Write your
new code
here. →

.....

.....

.....

.....

.....

.....

.....

.....

.....

there are no Dumb Questions

Q: Are there many of these BIFs in Python?

A: Yes. At the last count, there were over 70 BIFs in Python 3.

Q: Over 70! How am I to remember that many, let alone find out what they all are?

A: You don't have to worry about remembering. Let Python do it for you.

Q: How?

A: At the Python or IDLE shell, type `dir(__builtins__)` to see a list of the built-in stuff that comes with Python (that's two leading and trailing underscore characters, by the way). The shell spits out a big list. Try it. All those lowercase words are BIFs. To find out what any BIF does—like `input()`, for example—type `help(input)` at the shell for a description of the BIFs function.

Q: Why so many BIFs?

A: Why not? Because Python comes with lots of built-in functionality, it can mean less code for you to write. This Python philosophy is known as “batteries included”: there's enough included with Python to let you do most things well, without having to rely on code from third parties to get going. As well as lots of BIFs, you'll find that Python's standard library is rich and packed with features waiting to be exploited by you.



Exercise Solution

Here's a copy of the current list-processing code. Your task was to rewrite this code using an if statement and the `isinstance()` BIF to process a list that displays another list.

```
for each_item in movies:
    print(each_item)
```

Process the "movies" list as before.

for each_item in movies:

You need to check if the current item is a list.

if isinstance(each_item, list):

The inner loop needs a new target identifier.

If it is a list, use another "for" loop to process the nested list.

for nested_item in each_item:
print(nested_item)

else:

Did you manage to get your indentation right?

print(each_item)

If the current item of the enclosing list isn't a list, display it on screen.



An IDLE Session

Let's use IDLE to see if this code makes a difference to the output displayed on screen:

```
>>> for each_item in movies:
    if isinstance(each_item, list):
        for nested_item in each_item:
            print(nested_item)
    else:
        print(each_item)
```

The Holy Grail

1975

Terry Jones & Terry Gilliam

91

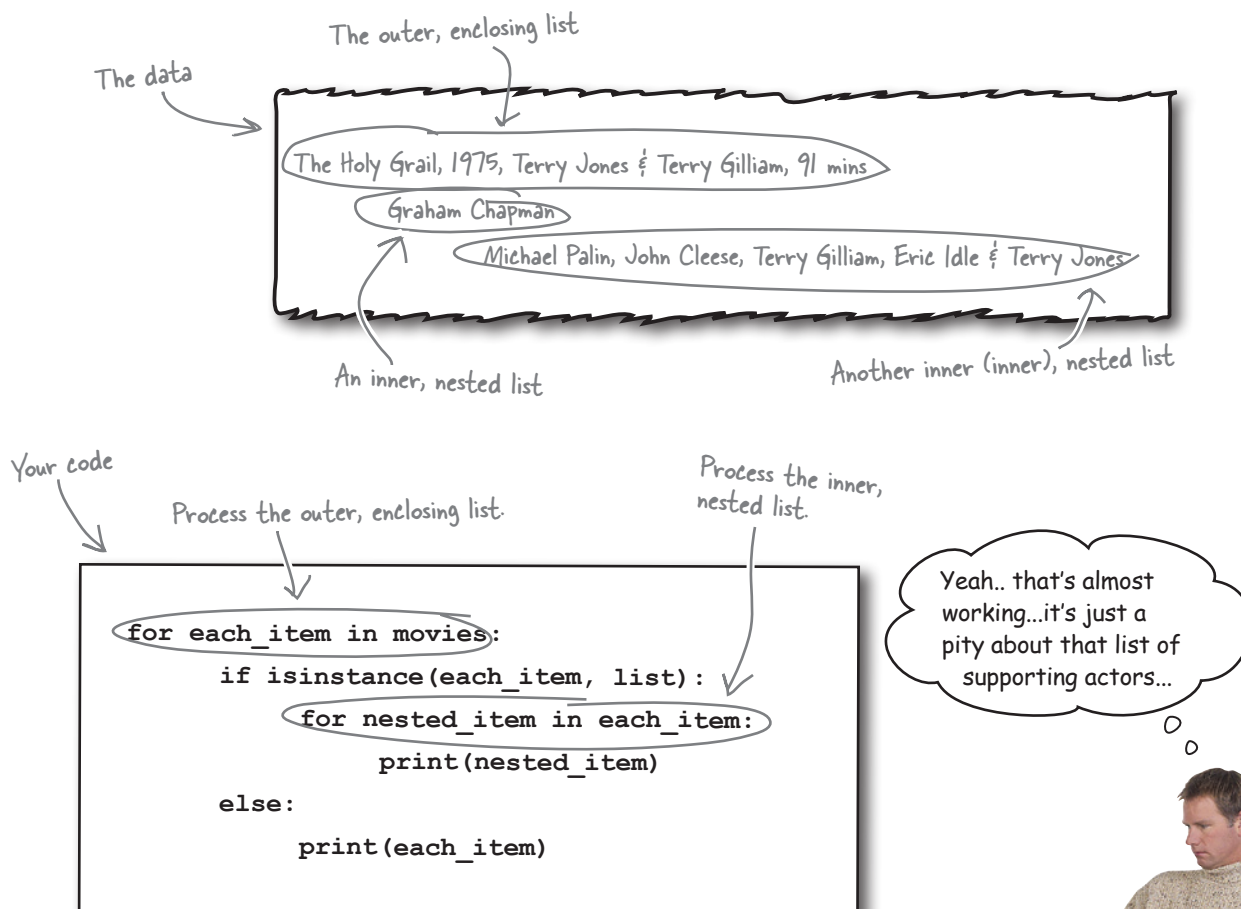
Graham Chapman

['Michael Palin', 'John Cleese', 'Terry Gilliam', 'Eric Idle', 'Terry Jones']

This is a little better, but not by much...there's another nested list here that's not being processed properly.

Complex data is hard to process

The movie buff's data is *complex*. Let's take another look at a subset of the data and your Python code that processes it.



Can you spot the problem with your Python code as it is currently written? What do you think needs to happen to your code to allow it to process the movie buff's data correctly?

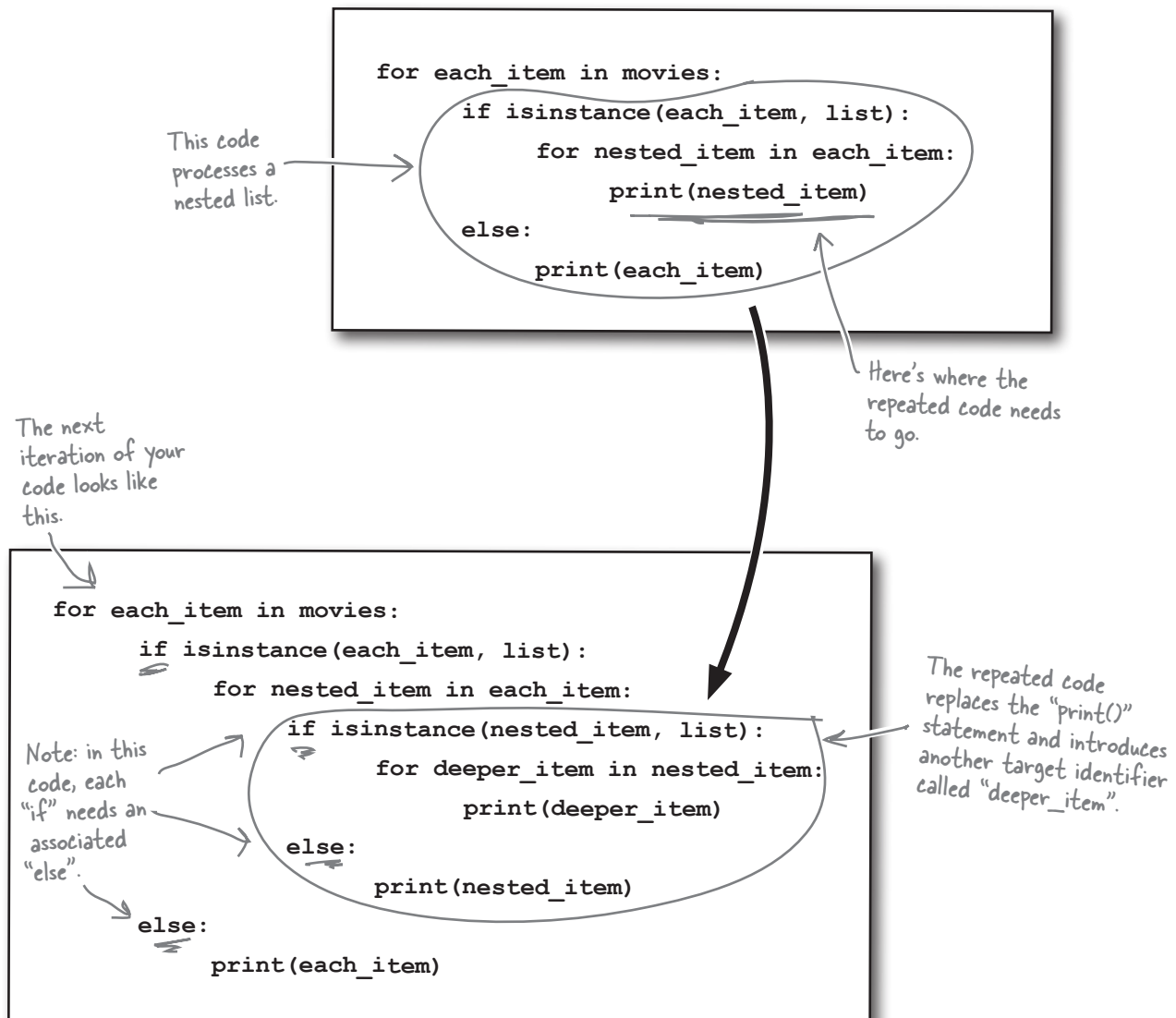


Handle many levels of nested lists

The data and your code are not in sync.

The movie buff's data is a list that contains a nested list that itself contains a nested list. The trouble is that your code knows only how to process a list nested inside an enclosing list.

The solution, of course, is to *add more code to handle the additionally nested list*. By looking at the existing code, it's easy to spot the code you need to repeat:





An IDLE Session

Let's use IDLE once more to test this latest iteration of your code:

```
>>> for each_item in movies:
    if isinstance(each_item, list):
        for nested_item in each_item:
            if isinstance(nested_item, list):
                for deeper_item in nested_item:
                    print(deeper_item)
            else:
                print(nested_item)
    else:
        print(each_item)
```

Process a deeply nested list inside a nested list inside an enclosing list.

The Holy Grail

1975

Terry Jones & Terry Gilliam

91

Graham Chapman

Michael Palin

John Cleese

Terry Gilliam

Eric Idle

Terry Jones

It works! This time, you see all of your list data on screen.

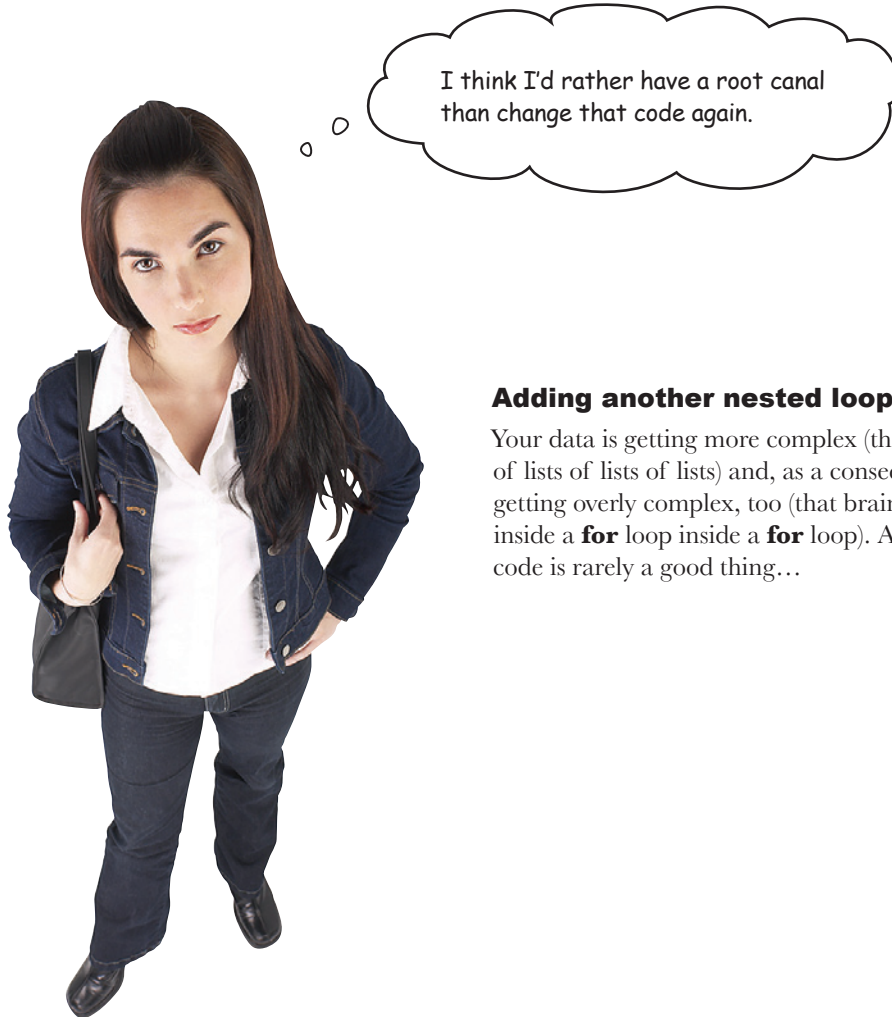


I just love that...in fact, I love it so much I've decided to add another list to my data. I want to include the other movies each supporting actor has starred in. If I add the data, can you change your code to print this data, too?

That's more list data and more Python code.

The data has to be embedded as *another* nested list within the already deeply nested list of supporting actors. That's possible to do, even though it makes your head hurt just to think about a list of lists of lists of lists! Amending your code is just a matter of adding another **for** loop and an **if** statement.

That doesn't sound like too much trouble, does it?



Adding another nested loop is a huge pain.

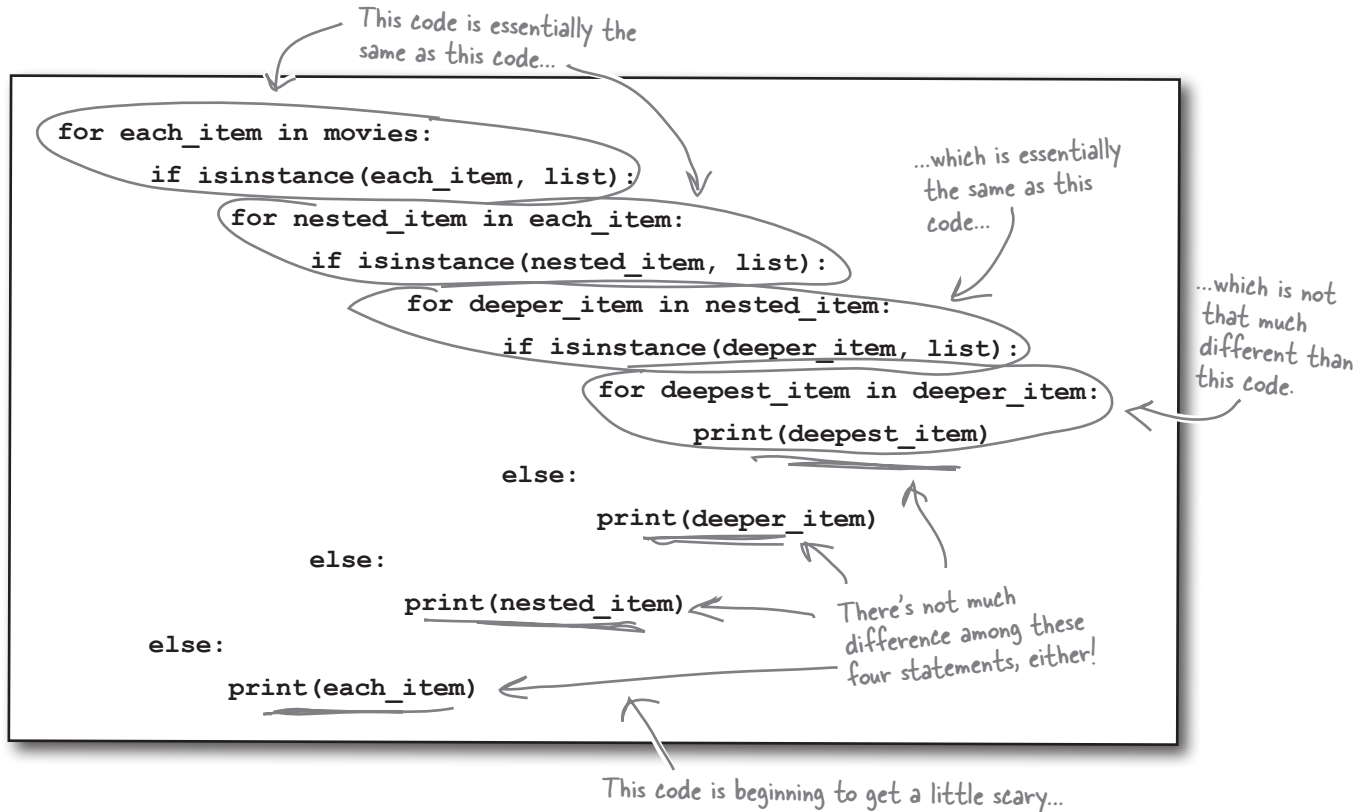
Your data is getting more complex (that mind-bending list of lists of lists of lists) and, as a consequence, your code is getting overly complex, too (that brain-exploding **for** loop inside a **for** loop inside a **for** loop). And overly complex code is rarely a good thing...



Wouldn't it be dreamy if there were an efficient way to process lists, preferably using a technique that resulted in less code, not more? But I know it's just a fantasy...

Don't repeat code; create a function

Take a look at the code that you've created so far, which (in an effort to save you from having *your* brain explode) has already been amended to process *yet another* nested list. Notice anything?



Your code now contains a lot of *repeated code*. It's also a mess to look at, even though it works with the movie buff's amended data. All that nesting of **for** loops is hard to read, and it's even harder to ensure that the **else** suites are associated with the correct **if** statement.

There has to be a better way...but what to do?

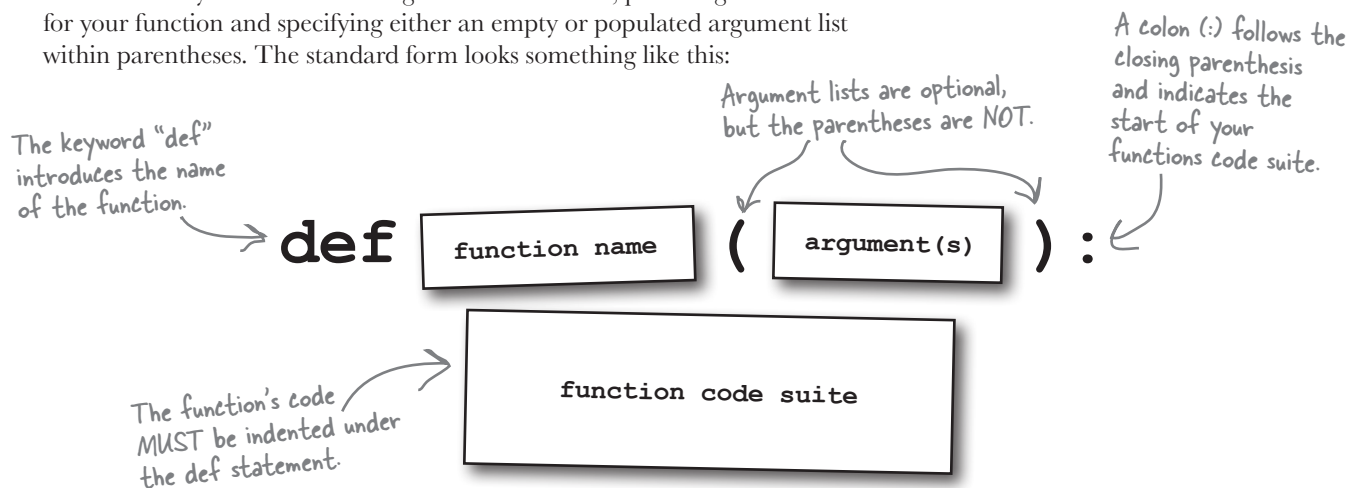
When code repeats in this way, most programmers look for a way to take the general pattern of the code and turn it into a **reusable function**. And Python programmers think this way, too. Creating a reusable function lets you invoke the function as needed, as opposed to cutting and pasting existing code.

So, let's turn the repeating code into a function.

Create a function in Python

A function in Python is a *named suite of code*, which can also take an **optional** list of arguments if required.

You define a Python function using the **def** statement, providing a name for your function and specifying either an empty or populated argument list within parentheses. The standard form looks something like this:



What does your function need to do?

Your function needs to take a list and process each item in the list. If it finds a nested list within the first list, the function needs to **repeat**. It can do this by *invoking itself* on the nested list. In other words, the function needs to **recur**—that is, *invoke itself* from within the function code suite.



Sharpen your pencil

Let's call the function that you'll create `print_lol()`. It takes one argument: a list to display on screen. Grab your pencil and complete the code below to provide the required functionality:

```
def print_lol(the_list):

    for .....
        if .....
            .....

        else:
            .....
```



Sharpen your pencil Solution

You were to call the function that you'll create `print_lol()`. It takes one argument: a list to display on screen. You were to grab your pencil and complete the code to provide the required functionality:

```
def print_lol(the_list):
```

Process the provided list with a "for" loop.

for each_item in the_list:

if isinstance(each_item, list):

If the item being processed is itself a list, invoke the function.

print_lol(each_item)

else:

print(each_item)

If the item being processed ISN'T a list, display the item on screen.



An IDLE Session

Let's use IDLE one final time to test your new function. Will it work as well as your earlier code?

```
>>> def print_lol(the_list):
```

```
    for each_item in the_list:
```

```
        if isinstance(each_item, list):
```

```
            print_lol(each_item)
```

```
        else:
```

```
            print(each_item)
```

← Define the function.

```
>>> print_lol(movies)
```

← Invoke the function.

```
The Holy Grail
```

```
1975
```

```
Terry Jones & Terry Gilliam
```

```
91
```

```
Graham Chapman
```

```
Michael Palin
```

```
John Cleese
```

```
Terry Gilliam
```

```
Eric Idle
```

```
Terry Jones
```

← It works, too! The recursive function produces EXACTLY the same results as the earlier code.

Recursion to the rescue!

The use of a *recursive function* has allowed you to **reduce** 14 lines of messy, hard-to-understand, brain-hurting code into a *six-line function*. Unlike the earlier code that needs to be amended to support additional nested lists (should the movie buff require them), the recursive function *does not need to change* to process any depth of nested lists properly.

Python 3 defaults its *recursion limit* to 1,000, which is a lot of lists of lists of lists of lists...and this limit can be changed should you ever need even more depth than that.



What a great start!

By taking advantage of functions and recursion, you've solved the code complexity problems that had crept into your earlier list-processing code.

By creating `print_lol()`, you've produced a reusable chunk of code that can be put to use in many places in your (and others) programs.

You're well on your way to putting Python to work!



Your Python Toolbox

You've got Chapter 1 under your belt and you've added some key Python goodies to your toolbox.

Python Lingo

- "BIF" – a built-in function.
- "Suite" – a block of Python code, which is indented to indicate grouping.
- "Batteries included" – a way of referring to the fact that Python comes with most everything you'll need to get going quickly and productively.

IDLE Notes

- The IDLE shell lets you experiment with your code as you write it.
- Adjust IDLE's preferences to suit the way you work.
- Remember: when working with the shell, use **Alt-P** for Previous and use **Alt-N** for Next (but use **Ctrl** if you're on a Mac).



BULLET POINTS

- Run Python 3 from the command line or from within IDLE.
- Identifiers are names that refer to data objects. The identifiers have no "type," but the data objects that they refer to *do*.
- `print()` BIF displays a message on screen.
- A **list** is a collection of data, separated by commas and surrounded by square brackets.
- Lists are like arrays on steroids.
- Lists can be used with BIFs, but also support a bunch of list methods.
- Lists can hold any data, and the data can be of mixed type. Lists can also hold other lists.
- Lists shrink and grow as needed. All of the memory used by your data is managed by Python for you.
- Python uses indentation to group statements together.
- `len()` BIF provides a length of some data object or count the number of items in a collection, such as a list.
- The **for** loop lets you iterate a list and is often more convenient to use than an equivalent **while** loop.
- The **if... else...** statement lets you make decisions in your code.
- `isinstance()` BIF checks whether an identifier refers to a data object of some specified type.
- Use **def** to define a custom function.

2 sharing your code

Modules of functions



Reusable code is great, but a shareable module is better.

By sharing your code as a Python module, you open up your code to the entire Python community...and it's always good to share, isn't it? In this chapter, you'll learn how to create, install, and distribute your own shareable modules. You'll then load your module onto Python's software sharing site on the Web, so that *everyone* can benefit from your work. Along the way, you'll pick up a few new tricks relating to Python's functions, too.

It's too good not to share

You've been showing your function to other programmers, and they like what they see.



Yes, a function this good should be shared with the world.

Python provides a set of technologies that make this easy for you, which includes *modules* and *the distribution utilities*:

- **Modules let you organize your code for optimal sharing.**
- **The distribution utilities let you share your modules with the world.**

Let's turn your function into a module, then use the distribution utilities to share your module with the wider Python programming community.

Turn your function into a module

A **module** is simply a text file that contains Python code. The main requirement is that the name of the file needs to end in `.py`: the Python extension. To turn your function into a module, save your code into an appropriately named file:

Your code
from
Chapter 1

```
def print_lol(the_list):
    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item)
        else:
            print(each_item)
```

Let's call
this file
"nester.py".



there are no
Dumb Questions

Q: What's the best Python editor?

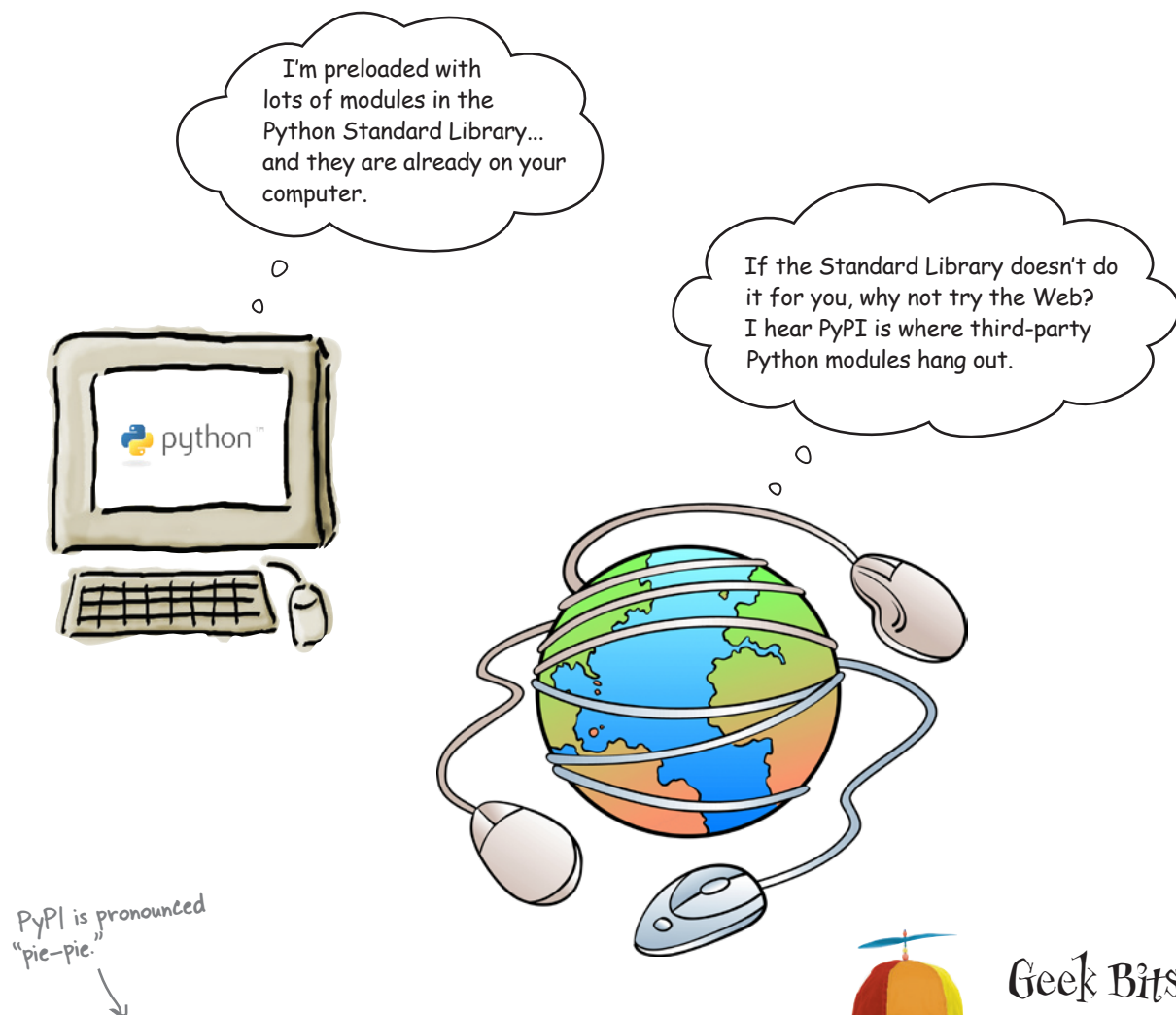
A: The answer to that question really depends on who you ask. However, you can, of course, use *any* text editor to create and save your function's code in a text file. Something as simple as NotePad on Windows works fine for this, as does a full-featured editor such as TextMate on Mac OS X. And there's also full-fledged IDEs such as Eclipse on Linux, as well as the classic vi and emacs editors. And, as you already know, Python comes with IDLE, which also includes a built-in code editor. It might not be as capable as those other "real" editors, but IDLE is installed with Python and is essentially guaranteed to be available. For lots of jobs, IDLE's edit window is all the editor you'll ever need when working with your Python code. Of course, there are other IDEs for Python, too. Check out WingIDE for one that specifically targets Python developers.

Do this!

Go ahead and create a text file called `nester.py` that contains your function code from the end of Chapter 1.

Modules are everywhere

As might be expected, you'll find Python modules in lots of places.



The **Python Package Index** (or *PyPI* for short) provides a centralized repository for third-party Python modules on the Internet. When you are ready, you'll use PyPI to publish your module and make your code available for use by others. And your module *is* ready, *but* for one important addition.

What do you think is missing from your module?



Geek Bits

If you are already familiar with Perl's CPAN repository, you can think of PyPI as the Python equivalent.

Comment your code

It's always a good idea to include comments with your code. As you plan to share your module with the world, well-written comments help to document your work.

In Python, a common commenting technique is to use a **triple quote** for multiple-line comments. When you use a triple quote without assigning it to a variable, everything *between* the triple quotes is considered a comment:

Start with a triple quote...

```
"""This is the standard way to
include a multiple-line comment in
your code."""
```

Hello! I'm a big string who just happens to be a Python comment, too. Nice, eh?

...and end with a triple quote.

Sharpen your pencil

Put your module comment here.

Here is your module code (which is saved in the file `nester.py`). In the spaces provided, use your pencil to compose two comments: the first to describe the module and the second to describe the function.

```
def print_lol(the_list):
```

Add a comment for your function here.

```
for each_item in the_list:
    if isinstance(each_item, list):
        print_lol(each_item)
    else:
        print(each_item)
```

Sharpen your pencil Solution



Here is your module code (which is saved in the file `nester.py`). In the spaces provided, you were asked to use your pencil to compose two comments: the first to describe the module and the second to describe the function.

"""This is the "nester.py" module, and it provides one function called `print_lol()` which prints lists that may or may not include nested lists."""

Did you remember to include the triple quotes?

```
def print_lol(the_list):
```

"""This function takes a positional argument called "the_list", which is any Python list (of, possibly, nested lists). Each data item in the provided list is (recursively) printed to the screen on its own line."""

```
for each_item in the_list:
```

```
    if isinstance(each_item, list):
```

```
        print_lol(each_item)
```

```
    else:
```

```
        print(each_item)
```

There are no changes to the actual code here; you're just adding some comments.

there are no Dumb Questions

Q: How do I know where the Python modules are on my computer?

A: Ask IDLE. Type `import sys; sys.path` (all on one line) into the IDLE prompt to see the list of locations that your Python interpreter searches for modules.

Q: Hang on a second. I can use ";" to put more than one line of code on the same line in my Python programs?

A: Yes, you can. However, I don't recommend that you do so. Better to give each Python statement its own line; it makes your code much easier for you (and others) to read.

Q: Does it matter where I put my `nester.py` module?

A: For now, no. Just be sure to put it somewhere where you can find it later. In a while, you'll install your module into your local copy of Python, so that the interpreter can find it without you having to remember when you actually put it.

Q: So comments are like a funny-looking string surrounded by quotes?

A: Yes. When a triple-quoted string is not assigned to a variable, it's treated like a comment. The comments in your code are surrounded by three double quotes, but you could have used single quotes, too.

Q: Is there any other way to add a comment to Python code?

A: Yes. If you put a "#" symbol anywhere on a line, everything from that point to the end of the current line is a comment (unless the "#" appears within a triple quote, in which case it's part of *that* comment). A lot of Python programmers use the "#" symbol to quickly switch on and off a single line of code when testing new functionality.



An IDLE Session

Now that you've added your comments and created a module, let's test that your code is still working properly. Rather than typing your function's code into IDLE's prompt, bring the `nester.py` file into IDLE's edit window, and then press F5 to run the module's code:

Note that the comments are color coded.

```
nester.py - /Users/barryp/HeadFirstPython/chapter2/nester.py

"""This is the "nester.py" module and it provides one function called print_lol()
   which prints lists that may or may not include nested lists."""

def print_lol(the_list):
    """This function takes one positional argument called "the_list", which
    is any Python list (of - possibly - nested lists). Each data item in the
    provided list is (recursively) printed to the screen on it's own line."""

    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item)
        else:
            print(each_item)
```

Nothing appears to happen, other than the Python shell “restarting” and an empty prompt appearing:

```
>>> ===== RESTART =====
>>>
>>>
```

What's happened is that the Python interpreter has reset and the code in your module has executed. The code defines the function but, other than that, does little else. The interpreter is patiently waiting for you to do something with your newly defined function, so let's create a list of lists and invoke the function on it:

```
>>> movies = [
    "The Holy Grail", 1975, "Terry Jones & Terry Gilliam", 91,
    ["Graham Chapman",
     ["Michael Palin", "John Cleese", "Terry Gilliam", "Eric Idle", "Terry Jones"]]]
```

Define the list of movies facts from Chapter 1.

```
>>> print_lol(movies) ← Invoke the function on the list.
```

```
The Holy Grail
```

```
1975
```

```
Terry Jones & Terry Gilliam
```

```
91
```

```
Graham Chapman
```

```
Michael Palin
```

```
John Cleese
```

```
Terry Gilliam
```

```
Eric Idle
```

```
Terry Jones
```

Cool. Your code continues to function as expected. The data in the list of lists is displayed on screen.

Prepare your distribution

In order to share your newly created module, you need to prepare a **distribution**. This is the Python name given to the collection of files that together allow you to build, package, and distribute your module.

Once a distribution exists, you can install your module into your local copy of Python, as well as upload your module to PyPI to share with the world. Follow along with the process described on these two pages to create a distribution for your module.

1 Begin by creating a folder for your module.

With the folder created, copy your `nester.py` module file into the folder. To keep things simple, let's call the folder `nester`:

The "nester.py"
module file.



Do this!

Follow along with each of the steps described on these pages. By the time you reach the end, your module will have transformed into a Python distribution.

The newly created
"nester" folder (or
directory).

2 Create a file called "setup.py" in your new folder.

This file contains metadata about your distribution. Edit this file by adding the following code:

Import the
"setup" function
from Python's
distribution utilities.

```
from distutils.core import setup
```

```
setup(
```

```
    name          = 'nester',
    version       = '1.0.0',
    py_modules    = ['nester'],
    author        = 'hfpypthon',
    author_email  = 'hfpypthon@headfirstlabs.com',
    url           = 'http://www.headfirstlabs.com',
    description   = 'A simple printer of nested lists',
)
```

Associate your module's
metadata with the setup
function's arguments.

These are the
setup function's
argument names.

These are the
values Head First
Labs use with
their modules;
your metadata
will be different.

Build your distribution

You now have a folder with two files in it: your module's code in `nester.py` and metadata about your module in `setup.py`. Now, it's time to build your distribution.

Note: if you are using Windows, replace "python3" in these commands with "c:\Python31\python.exe".

3 Build a distribution file.

The distribution utilities include all of the smarts required to build a distribution. Open a terminal window within your `nester` folder and type a single command: `python3 setup.py sdist`.

Enter the command at the prompt.

A collection of status messages appears on screen, confirming the creation of your distribution.

```
File Edit Window Help Build
$ python3 setup.py sdist
running sdist
running check
warning: sdist: manifest template 'MANIFEST.in' does not exist
warning: sdist: standard file not found: should have README

writing manifest file 'MANIFEST'
creating nester-1.0.0
making hard links in nester-1.0.0...
hard linking nester.py -> nester-1.0.0
hard linking setup.py -> nester-1.0.0
creating dist
Creating tar archive
removing 'nester-1.0.0' (and everything under it)
$
```

4 Install your distribution into your local copy of Python.

Staying in the terminal, type this command: `sudo python3 setup.py install`.

Another bunch of status messages appear on screen, confirming the installation of your distribution.

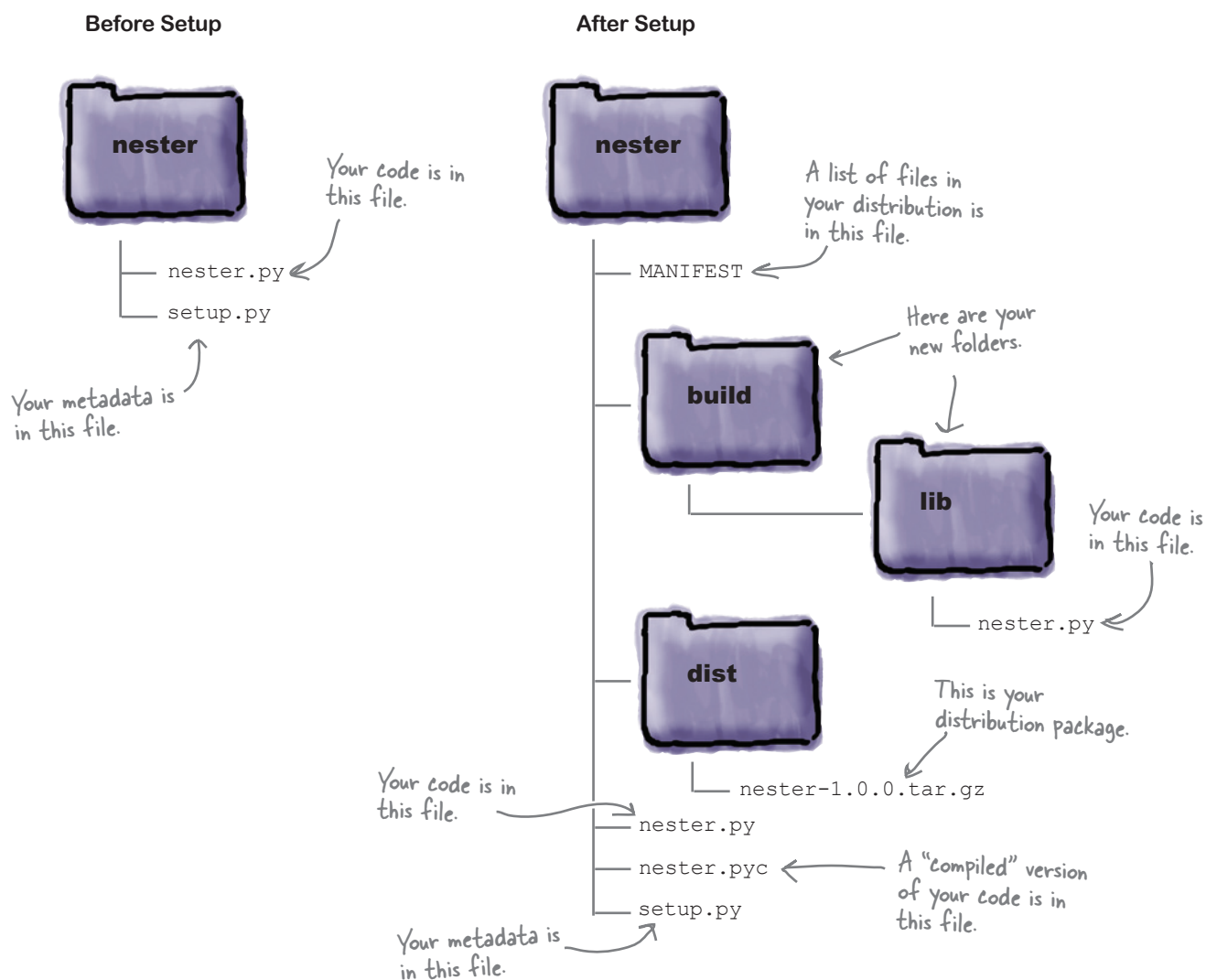
```
File Edit Window Help Install
$ python3 setup.py install
running install
running build
running build_py
creating build
creating build/lib
copying nester.py -> build/lib
running install_lib
copying build/lib/nester.py -> /Library/Frameworks/Python.framework/Versions/3.1/lib/python3.1/site-packages
byte-compiling /Library/Frameworks/Python.framework/Versions/3.1/lib/python3.1/site-packages/nester.py to nester.pyc
running install_egg_info
Writing /Library/Frameworks/Python.framework/Versions/3.1/lib/python3.1/site-packages/nester-1.0.0-py3.1.egg-info
```

Your distribution is ready.

A quick review of your distribution

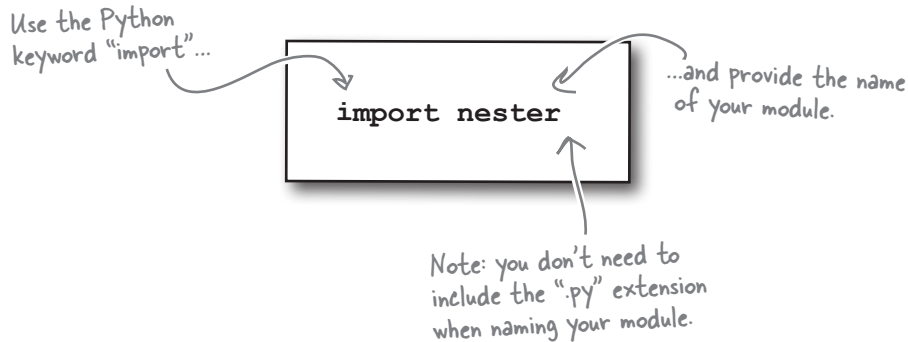
Thanks to Python's distribution utilities, your module has been *transformed* into a distribution and *installed* into your local copy of Python.

You started with a single function, which you entered into a file called `nester.py`, creating a module. You then created a folder called `nester` to house your module. The addition of a file called `setup.py` to your folder allowed you to build and install your distribution, which has resulted in a number of additional files and two new folders appearing within your `nester` folder. These files and folders are all created for you by the distribution utilities.



Import a module to use it

Now that your module is built, packaged as a distribution, and installed, let's see what's involved in using it. To use a module, simply `import` it into your programs or import it into the IDLE shell:



The `import` statement tells Python to include the `nester.py` module in your program. From that point on, you can use the module's functions as if they were entered directly into your program, right? Well...that's what you might expect. Let's check out the validity of your assumption.



Exercise

Write a small program that imports your newly created module, defines a small list called "cast," and then uses the function provided by your module to display the contents of the list on screen. Use the following list data (all strings): Palin, Cleese, Idle, Jones, Gilliam, and Chapman.

.....

.....

.....

.....

.....

.....

Open your program in IDLE's edit window, and then press F5 to execute your code. Describe what happens in the space below:

.....

.....

.....



Exercise Solution

You were to write a small program that imports your newly created module, defines a small list called "cast," and then uses the function provided by your module to display the contents of the list on screen. You were to use the following list data (all strings): Palin, Cleese, Idle, Jones, Gilliam, and Chapman.

It's a simple three-line program. There's nothing too difficult here.

```
import nester
```

```
cast = ['Palin', 'Cleese', 'Idle', 'Jones', 'Gilliam', 'Chapman']
```

```
print_lol(cast)
```

Open your program in IDLE's edit window, and then press F5 to execute your code. Describe what happens in the space below:

But it didn't work! → IDLE gives an error, and the program does not run!



An IDLE Session

With your program in the IDLE edit window, pressing F5 (or choosing Run Module from the Run menu) does indeed cause problems:

```
try_nester.py - /Users/barryp/HeadFirstPython/chapter2/try_nester.py

import nester
cast = ['Palin', 'Cleese', 'Idle', 'Jones', 'Gilliam', 'Chapman']
print_lol(cast)
```

Ln: 6 Col: 0

Your program does not appear to have executed and an error message is reported:

```
>>> ===== RESTART =====
>>>
Traceback (most recent call last):
  File "/Users/barryp/HeadFirstPython/chapter2/try_nester.py", line 4, in <module>
    print_lol(cast)
NameError: name 'print_lol' is not defined
>>>
```

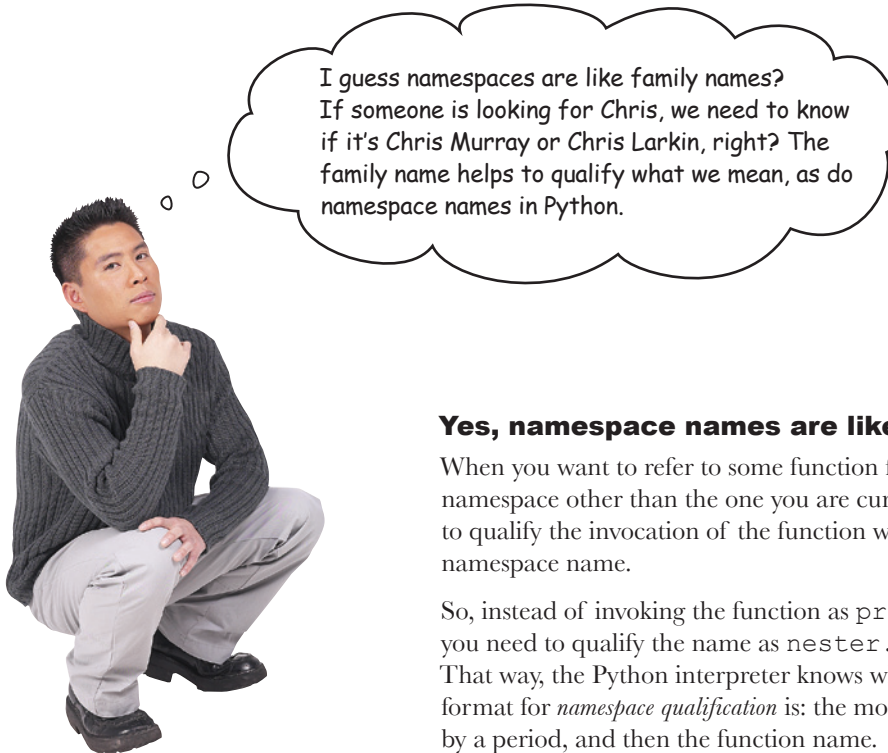
With your program in IDLE, pressing F5 causes a NameError...it looks like your function can't be found!!!

Python's modules implement namespaces

All code in Python is associated with a *namespace*.

Code in your main Python program (and within IDLE's shell) is associated with a namespace called `__main__`. When you put your code into its own module, Python automatically creates a namespace with the same name as your module. So, the code in your module is associated with a namespace called `nester`.

← That's a double underscore in front of the word "main" and after it.



Yes, namespace names are like family names.

When you want to refer to some function from a module namespace other than the one you are currently in, you need to qualify the invocation of the function with the module's namespace name.

So, instead of invoking the function as `print_lol(cast)` you need to qualify the name as `nester.print_lol(cast)`. That way, the Python interpreter knows where to look. The format for *namespace qualification* is: the module's name, followed by a period, and then the function name.

The module name, which identifies the namespace.

`nester.print_lol(cast)`

The function is then invoked as normal, with "cast" provided as the list to process.

A period separates the module namespace name from the function name.



An IDLE Session

Let's test this. Staying at the IDLE shell, import your module, create the list, and then try to invoke the function without a qualifying name. You're expecting to see an error message:

```
>>> import nester
>>> cast = ['Palin', 'Cleese', 'Idle', 'Jones', 'Gilliam', 'Chapman']
>>> print_lol(cast)
```

Traceback (most recent call last):

File "<pyshell#4>", line 1, in <module>

print_lol(cast)

NameError: name 'print_lol' is not defined

As expected, your code has caused a NameError, because you didn't qualify the name.

When you qualify the name of the function with the namespace, things improve dramatically:

```
>>> nester.print_lol(cast)
```

Palin

Cleese

Idle

Jones

Gilliam

Chapman

This time, things work as expected...the data items in the list are displayed on screen.



Geek Bits

When you use a plain import statement, such as `import nester`, the Python interpreter is instructed to allow you to access `nester`'s functions using namespace qualification. However, it is possible to be *more specific*. If you use `from nester import print_lol`, the specified function (`print_lol` in this case) is *added* to the current namespace, effectively *removing* the requirement for you to use namespace qualification. But you need to be **careful**. If you already have a function called `print_lol` defined in your current namespace, the specific import statement overwrites your function with the imported one, which might not be the behavior you want.

Your module is now ready for upload to PyPI.

Register with the PyPI website

In order to upload your distribution to PyPI, you need to register with the PyPI website. This is a relatively straightforward process.

Begin by surfing over to the PyPI website at <http://pypi.python.org/> and requesting a PyPI ID:

Provide the Username you'd like to use

Don't try to use "hfpypthon," because that Username is already taken.

Enter your chosen password twice for confirmation purposes.

Don't worry about providing a PGP key (unless you actually have one).

Provide a valid email address.

Don't forget to click the "I agree" checkbox before clicking on the Register button.

Manual user registration

Username:

Password:

Confirm:

Email Address:

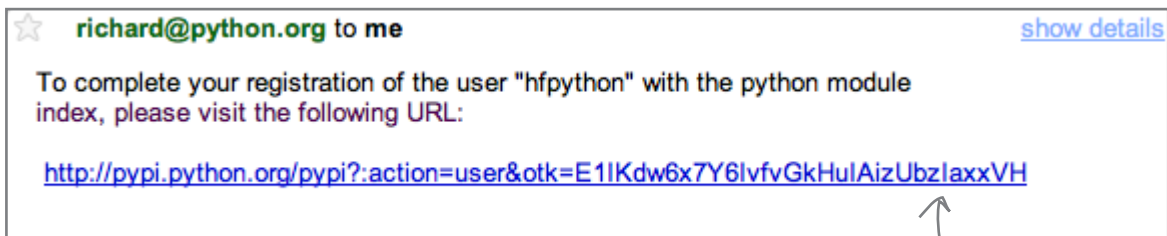
PGP Key ID (optional):

Usage Agreement: ☐ I agree

By registering to upload content to PyPI, I agree and affirmatively acknowledge the following:

1. Content is restricted to Python packages and related information only.
2. Any content uploaded to PyPI is provided on a non-confidential basis.
3. The PSF is free to use or disseminate any content that I upload on an unrestricted basis. The content on the web site are granted an irrevocable, worldwide, royalty-free, nonexclusive license to the PSF, including in digital form.
4. I represent and warrant that I have complied with all government regulations concerning the export of technology, and in particular, if I am subject to United States law, I represent and warrant that I have obtained all necessary export permits for the content I upload. I further affirm that any content I provide is not intended for use by any entity on the United States Export Administration Regulations.

If all of your registration details are in order, a confirmation message is sent to the email address submitted on the registration form. The email message contains a link you can click to confirm your PyPI registration:



You are now registered with PyPI.

Click the confirmation link to complete your PyPI registration.

Upload your code to PyPI

You're ready to rock! The code in your function has been placed in a module, used to create a distribution and installed into your local copy of Python. To upload your distribution to PyPI, complete these two steps: command-line *registration* with PyPI and command-line *uploading*.

It might seem strange to have to register with PyPI *again*, seeing as you just did this with their website. However, the command-line uploading tool needs to be made aware of your PyPI Username and Password, and that's what this registration does. Don't worry: you have to do this only once.

Instruct setup to register your details.

Confirm that you want to use your just-created PyPI credentials.

Use your PyPI settings and save them for future use.

```
File Edit Window Help Register
$ python3 setup.py register
running register
running check
We need to know who you are, so please choose either:
  1. use your existing login,
  2. register as a new user,
  3. have the server generate a new password for you (and email it to you), or
  4. quit
Your selection [default 1]:
1
Username: hfpypthon
Password:
Registering nester to http://pypi.python.org/pypi
Server response (200): OK
I can store your PyPI login so future submissions will be faster.
(the login will be stored in /Users/barryp/.pypirc)
Save your login (y/N)?y
```

With your registration details entered and saved, you are now ready to upload your distribution to PyPI. Another command line does the trick:

Note: If you try to upload a module called "nester", you'll get an error as that name's already taken. ☺

Instruct setup to upload your software distribution to PyPI.

Setup confirms that the upload is successful. Your distribution is now part of PyPI.

```
File Edit Window Help Upload
$ python3 setup.py sdist upload
running sdist
running check
reading manifest file 'MANIFEST'
creating nester-1.0.0
making hard links in nester-1.0.0...
hard linking nester.py -> nester-1.0.0
hard linking setup.py -> nester-1.0.0
Creating tar archive
removing 'nester-1.0.0' (and everything under it)
running upload
Submitting dist/nester-1.0.0.tar.gz to http://pypi.python.org/pypi
Server response (200): OK
$
```


Welcome to the PyPI community

Congratulations! You are now a full-fledged, card-carrying member of the PyPI community. Your distribution has joined the over 10,000 other uploads on PyPI. Feel free to surf on over to the PyPI website to confirm the upload.

Programmers from all over the globe are now able to download, unpack, and install your module into their local copy of Python, which is pretty cool when you think about it.

Sit back, put your feet up, and wait for the plaudits to begin...

You've now written and published your code... how cool is that?

there are no Dumb Questions

Q: Which is best: plain imports or specific imports?

A: Neither, really. Most programmers mix and match based on their own personal preference and taste (although there are plenty of programmers willing to argue that their preferred way is the “one true way”).

Note that the `from module import function` form pollutes your current namespace: names already defined in your current namespace are *overwritten* by the imported names.

Q: And when I press F5 in IDLE's edit window, it's as if the module's code is imported with an `import` statement, right?

A: Yes, that is essentially what happens. The code in your edit window is compiled and executed by Python, and any names in the edit window are imported into the namespace being used by IDLE's shell. This is handy, because it makes it easy to test functionality with IDLE. But bear in mind that outside of IDLE, you still need to import your module *before* you can use its functionality.

Q: Is it really necessary for me to install my modules into my local copy of Python? Can't I just put them in any old folder and import them from there?

A: Yes, it is possible. Just bear in mind that Python looks for modules in a very specific list of places (recall the `import sys; sys.path` trick from earlier in this chapter). If you put your modules in a folder not listed in Python's path list, chances are the interpreter won't find them, resulting in `ImportErrors`. Using the distribution utilities to build and install your module into your local copy of Python avoids these types of errors.

Q: I noticed the distribution utilities created a file called `nester.pyc`. What's up with that?

A: That's a very good question. When the interpreter executes your module code for the first time, it reads in the code and translates it into an internal bytecode format which is ultimately executed. (This idea is very similar to the way the Java JVM works: your Java code is turned into a class file as

a result of your Java technologies compiling your code.) The Python interpreter is smart enough to skip the translation phase the next time your module is used, because it can determine when you've made changes to the original module code file. If your module code hasn't changed, no translation occurs and the “compiled” code is executed. If your code has changed, the translation occurs (creating a new `pyc` file) as needed. The upshot of all this is that when Python sees a `pyc` file, it tries to use it because doing so *makes everything go much faster*.

Q: Cool. So I can just provide my users with the `pyc` file?

A: No, don't do that, because the use of the `pyc` file (if found) is primarily a runtime optimization performed by the interpreter.

Q: So, can I delete the `pyc` file if I don't need it?

A: Sure, if you really want to. Just be aware that you lose any potential runtime optimization.

With success comes responsibility

Lots of programmers from many different locations are using your module.
And some of these programmers are looking for more features.



Requests for change are inevitable

You need to keep your current users happy by maintaining the existing functionality, while at the same time providing enhanced functionality to those users that require it. This could be tricky.

What are your options here?

Life's full of choices

When it comes to deciding what to do here, there's no shortage of suggestions.

That's soooo easy. Simply create another function called "print_lol2", right? You could then import the function you want using the specific form of the import statement. It's not that hard, really...

Yeah, that might just work.

You could edit your module's code and define a new function called `print_lol2`, then code up the function to perform the nested printing. When you want to use the original function, use this specific form of the import statement: `from nester import print_lol`. When you want to use the new, improved version of the function, use this import statement: `from nester import print_lol2`.

Which would work, but...

But that suggestion is twice the work...which might be OK sometimes...but the creation of a second, almost identical, function seems wasteful to me.



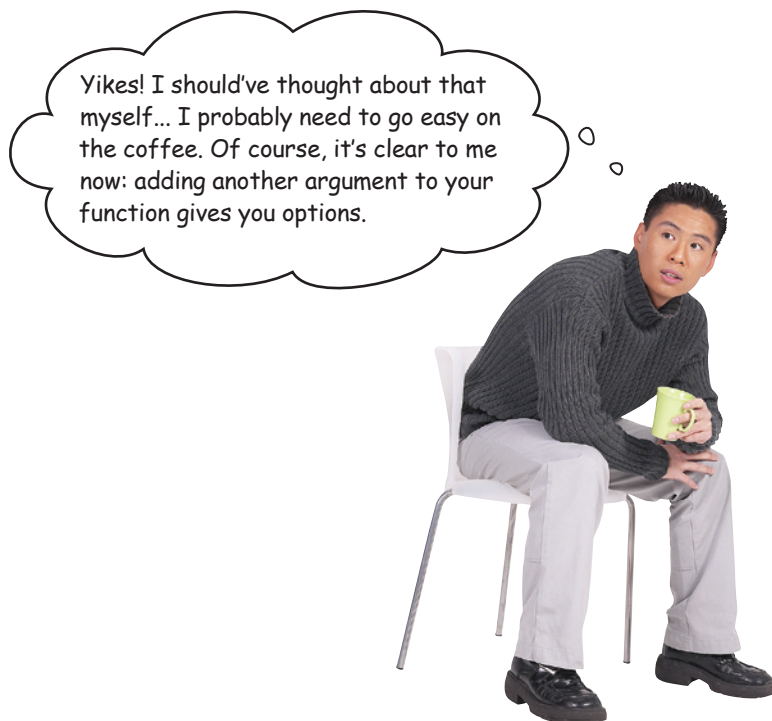
Right. A second function is wasteful.

Not only are you introducing an almost identical function to your module, which might create a potential maintenance nightmare, but you're also making things much more difficult for the users of your module, who must decide *ahead of time* which version of the function they need. Adding a second function makes your module's *application programming interface* (API) more complex than it needs to be.

There has to be a better strategy, doesn't there?

Control behavior with an extra argument

If you add an extra argument to your function, you can handle indentation within your current code without too much trouble.



Take your function to the next level

At the moment, your function has a single argument: `the_list`. If you add a second argument called `level`, you can use it to control indentation. A positive value for `level` indicates the number of tab-stops to include when displaying a line of data on screen. If `level` is 0, no indentation is used; if it's 1, use a single tab-stop; if it's 2, use two tab-stops; and so on.

It's clear you are looking at some sort of looping mechanism here, right? You already know how to iterate over a variably sized list, but how do you iterate a fixed number of times in Python?

Does Python provide any functionality that can help?

Before you write new code, think BIF

When you come across a need that you think is *generic*, ask yourself if there's a built-in function (BIF) that can help. After all, iterating a fixed number of times is something you'll need to do *all the time*.

And remember: Python 3 includes over 70 BIFs, so there's a lot of functionality waiting to be discovered.



Use your pencil to draw a line matching each BIF to the correct description. The first one is done for you. Once you have all your lines drawn, circle the BIF you think you need to use in the next version of your function.

BIF

What the BIF does

`list()`

Creates a numbered list of paired-data, starting from 0.

`range()`

Returns the unique identification for a Python data object.

`enumerate()`

A factory function that creates a new, empty list.

`int()`

Returns the next item from a iterable data structure such as a list.

`id()`

Returns an iterator that generates numbers in a specified range on demand and as needed.

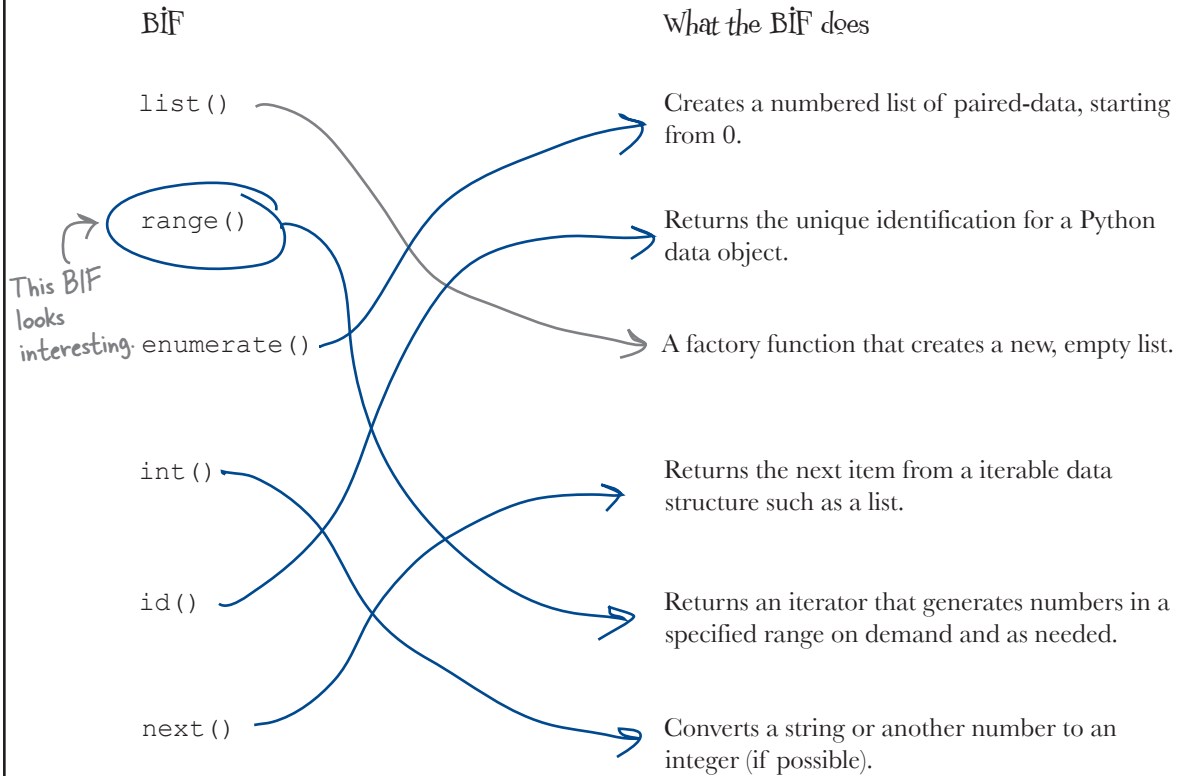
`next()`

Converts a string or another number to an integer (if possible).

WHO DOES WHAT?

SOLUTION

You were to use your pencil to draw a line matching each BIF to the correct description. Once you had all your lines drawn, you were to circle the BIF you think you need to use in the next version of your function.



The range() BIF iterates a fixed number of times

The range() BIF gives you the control you need to iterate a specific number of times and can be used to generate a list of numbers from zero up-to-but-not-including some number. Here's how to use it:

"num" is the target identifier and is assigned each of the numbers generated by "range()" in turn.

```
for num in range(4):
    print(num)
```

Generate numbers up-to-but-not-including 4.

The numbers 0, 1, 2, and 3 will appear on screen.

there are no Dumb Questions

Q: Don't I need to import the BIFs in order to use them in my program?

A: No. For all intents and purposes, the BIFs are specifically imported into every Python program as well as IDLE.

Q: So the BIFs must belong to the `__main__` namespace, right?

A: No. They are automatically imported into the `__main__` namespace, but the BIFs have their very own namespace called (wait for it) `__builtins__`.

Q: I get how `range()` works, but surely I could just as easily use a `while` loop to do the same thing?

A: Yes, you can, but it's not as elegant as using `range()`. Seriously, though, the `while` equivalent not only requires you to write more code, but it also makes it your responsibility to worry about loop state, whereas `range()` worries about this for you. As a general rule, Python programmers look for ways to reduce the amount of code they need to write and worry about, which leads to better code robustness, fewer errors, and a good night's sleep.

Q: So BIFs are actually good for me?

A: BIFs exist to make your programming experience as straightforward as possible by providing a collection of functions that provide common solutions to common problems. Since they are included with Python, you are pretty much assured that they have been tested to destruction and do "exactly what it says on the tin." You can depend on the BIFs. Using them gives your program a leg up and makes you look good. So, yes, the BIFs are good for you!



Exercise

Now that you know a bit about the `range()` BIF, amend your function to use `range()` to indent any nested lists a specific number of tab-stops.

Hint: To display a TAB character on screen using the `print()` BIF yet avoid taking a new-line (which is `print()`'s default behavior), use this Python code: `print("\t", end='')`.

```
"""This is the "nester.py" module and it provides one function called print_lol()
   which prints lists that may or may not include nested lists."""
```

```
def print_lol(the_list, .....):
```

Include the name of the extra argument.

```
    """This function takes a positional argument called "the_list", which
       is any Python list (of - possibly - nested lists). Each data item in the
       provided list is (recursively) printed to the screen on it's own line."""
```

```
    for each_item in the_list:
```

```
        if isinstance(each_item, list):
            print_lol(each_item)
```

```
        else:
```

```
            .....
```

```
            print(each_item)
```

Don't forget to edit the comment.

Add code here to take the required number of tab-stops.



Exercise Solution

Now that you know a bit about the `range()` BIF, you were to amend your function to use `range()` to indent any nested lists a specific number of tab-stops.

Hint: To display a TAB character on screen using the `print()` BIF yet avoid taking a new-line (which is `print()`'s default behavior), use this Python code: `print("\t", end='')`.

```
"""This is the "nester.py" module and it provides one function called print_lol()
which prints lists that may or may not include nested lists."""
```

```
def print_lol(the_list, level):
    """This function takes a positional argument called "the_list", which
    is any Python list (of - possibly - nested lists). Each data item in the
    provided list is (recursively) printed to the screen on it's own line.
    A second argument called "level" is used to insert tab-stops when a nested list is encountered.
    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, level + 1)
        else:
            for tab_stop in range(level):
                print("\t", end="")
            print(each_item)
```

Use the value of "level" to control how many tab-stops are used.

Display a TAB character for each level of indentation.



An IDLE Session

It's time to test the new version of your function. Load your module file into IDLE, press F5 to import the function into IDLE's namespace, and then invoke the function on your `movies` list with a second argument:

```
>>> print_lol(movies, 0)
The Holy Grail
1975
Terry Jones & Terry Gilliam
91
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    print_lol(movies,0)
  File "/Users/barryp/HeadFirstPython/chapter2/nester/nester.py", line 14, in print_lol
    print_lol(each_item)
TypeError: print_lol() takes exactly 2 positional arguments (1 given)
```

Invoke your function, being sure to provide a second argument.

The data in "movies" starts to appear on screen...

...then all hell breaks loose! Something is not right here.

Here's your clue as to what's gone wrong.

Your code has a `TypeError`, which caused it to crash.

Python tries its best to run your code

Unlike compiled languages (such as C, Java, C#, and others), Python doesn't *completely* check the validity of your code *until it runs*. This allows Python to do some rather cool things that just aren't possible in those other languages, such as dynamically defining functions at runtime. This, of course, can be very flexible and powerful.

The cost to you, however, is that you need to be very careful when writing your code, because something that typically would be caught and flagged as an “error” by a traditional, statically typed, compiled language often goes unnoticed in Python.



...OK, C++ syntax fine...continuing to parse...whoops! You're trying to use a function before it's declared?!? That's NOT allowed around here... I'm outta here.

Ah ha! The old “calling a function before you've defined it yet” trick, eh? I'll just make a note in case you define it later at runtime. You are planning to do that, right? Please don't disappoint me, or I'll give you an error...



Take another look at the error on the opposite page. Why do you think Python is giving you this particular error? What do you think is wrong?

Trace your code

When you're trying to work out what went wrong with a program that looks like it should be OK, a useful technique is to trace what happens as each line of code executes. Here's the code that you are currently working with. At only three lines long (remember: the creation of the list is one line of code), it doesn't look like it should cause any trouble:

```
import nester ← These two lines look OK.

movies = [ "The Holy Grail", 1975, "Terry Jones & Terry Gilliam",
           91, ["Graham Chapman", ["Michael Palin",
                                   "John Cleese", "Terry Gilliam", "Eric Idle",
                                   "Terry Jones"]] ]

nester.print_lol(movies, 0) ← You are invoking the function with two arguments,
                             so that's OK, too.
```

With the data assigned to the function's arguments, the function's code starts to execute on each data item contained within the passed-in list:

To save space, the entire comment is not shown.

Process each item in the list...

...then decide what to do next based on whether or not the data item is a list.

```
def print_lol(the_list, level):
    """This function ... """
    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item)
        else:
            for tab_stop in range(level):
                print("\t", end='')
            print(each_item)
```

The "movies" list is assigned to "the_list", and the value 0 is assigned to "level".

If the data item is a list, recursively invoke the function...hang on a second, that doesn't look right!?

Work out what's wrong

There's your problem: the recursive invocation of your function is using the old *function signature* that required only one argument. The new version of your function requires two arguments.

The fix is easy: provide the correct number of arguments when calling the new version of your function. So, this snippet of code from your function:

```
if isinstance(each_item, list):
    print_lol(each_item)
```

needs to be rewritten to specify the correct number of arguments:

```
if isinstance(each_item, list):
    print_lol(each_item, level)
```

Not so fast. Surely the nested list needs to be printed after a specific number of tab-stops? At the moment, your code sets "level" to 0 but never changes the value, so "level" never has any impact on your displayed output...



Right. Your use of "level" needs one final tweak.

The whole point of having `level` as an argument is to allow you to control the nested output. Each time you process a nested list, you need to increase the value of `level` by 1. Your code snippet needs to look like this:

```
if isinstance(each_item, list):
    print_lol(each_item, level+1)
```

Simply increment the value of `level` by 1 each time you recursively invoke your function.

It's time to perform that update.

Update PyPI with your new code

Go ahead and edit your `nester.py` module (in the `nester` folder) to invoke your function properly. Now that you have a new version of your module, it's a good idea to update the distribution that you uploaded to PyPI.

With your code amended, there's also a small change needed to your distribution's `setup.py` program. You've changed your API, so adjust the value associated with `version` in `setup.py`. Let's move from version 1.0.0 to 1.1.0:

```
from distutils.core import setup
```

```
setup(
```

```
    name          = 'nester',
```

```
    version       = '1.1.0',
```

```
    py_modules    = ['nester'],
```

```
    author        = 'hfpypthon',
```

```
    author_email  = 'hfpypthon@headfirstlabs.com',
```

```
    url           = 'http://www.headfirstlabs.com',
```

```
    description   = 'A simple printer of nested lists',
```

```
)
```

Change the value associated with "version" to indicate to PyPI that this is indeed a new version.

Just as you did when you created and uploaded your distribution, invoke the `setup.py` program within your distribution folder to perform the upload:

```
File Edit Window Help UploadAgain
```

```
$ python3 setup.py sdist upload
```

```
running sdist
```

```
running check
```

```
reading manifest file 'MANIFEST'
```

```
creating nester-1.1.0
```

```
making hard links in nester-1.1.0...
```

```
hard linking nester.py -> nester-1.1.0
```

```
hard linking setup.py -> nester-1.1.0
```

```
Creating tar archive
```

```
removing 'nester-1.1.0' (and everything under it)
```

```
running upload
```

```
Submitting dist/nester-1.1.0.tar.gz to http://pypi.python.org/pypi
```

```
Server response (200): OK
```

```
$
```

Don't you just love those "200 OK" messages?

Your new distribution is now available on PyPI.



Mark: Take a look at this, guys...the `nester` module has been updated on PyPI.

Bob: Version 1.1.0...

Laura: I wonder what's changed?

Mark: It still works with nested lists of lists, but now you can see the nested structure on screen, which I think is pretty cool.

Laura: And useful. I've been waiting for that feature.

Bob: Eh...OK...but how do I upgrade my existing local copy?

Mark: Just follow the same steps as when you downloaded and installed `nester` from PyPI the first time.

Bob: So I download the package file, unpack it, and ask `setup.py` to install it into my Python for me?

Mark: Yes. It couldn't be any easier.

Laura: And what about my existing version of `nester`; what happens to that "old" version?

Bob: Yeah...do I have two `nester` modules now?

Mark: No. When you use `setup.py` to install the latest version it becomes the current version and effectively replaces the previous module, which was the 1.0.0 release.

Bob: And PyPI knows to give you the latest version of the module, too, right?

Mark: Yes, when you surf the PyPI website and search for `nester`, you are always provided with the latest version of the module.

Laura: Well, I use this module all the time and I've been waiting for this feature. I think I'll update right away.

Mark: I've already upgraded mine, and it works a treat.

Bob: Yeah, I use it a lot, too, so I guess I'll keep my system up to date and install the latest version. It's probably not a good idea to rely on out-of-date software, right?

Mark: I'd say. And, there's nothing quite like progress.

Laura: Catch you later, guys, I've got work to do.

Bob: Me, too. I'm off to PyPI to grab the latest `nester` and install it into my local copy of Python. I'll give it a quick test to confirm all is OK.

Mark: Later, dudes...

You've changed your API

Your new version of `nester` is indeed better, but not for all your users.



In your rush to release the latest and greatest version of your module, you forgot about *some* of your existing users. Recall that not all of your users want the new nested printing feature. However, by adding the second argument to `print_lol()`, you've changed your function's signature, which means your module has a different API. Anyone using the old API is going to have problems.

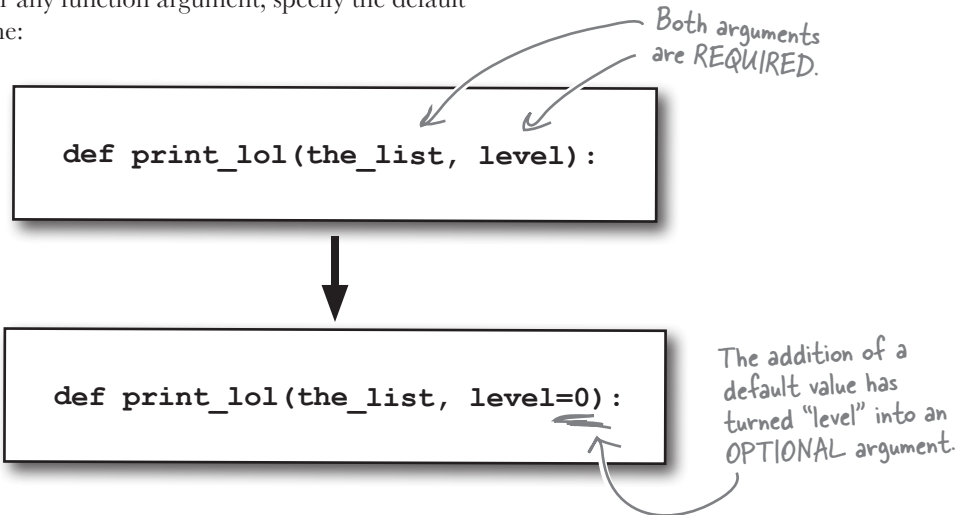
The ideal solution would be to provide **both** APIs, one which switches on the new feature and another that doesn't. Maybe the feature could be optional?

But how would that work?

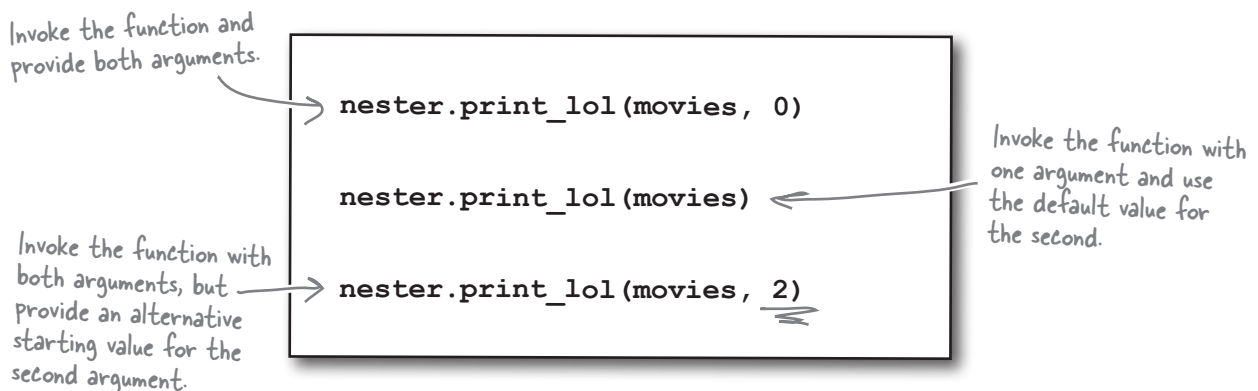
Use optional arguments

To turn a required argument to a function into an *optional* argument, provide the argument with a default value. When no argument value is provided, the default value is used. When an argument value is provided, it is used *instead* of the default. The key point is, of course, that the default value for the argument effectively makes the argument optional.

To provide a default value for any function argument, specify the default value *after* the argument name:



With the default value for the argument defined, you can now invoke the function in a number of different ways:



Your function now supports different signatures, but the functionality remains as it was.



An IDLE Session

Amend your code to give the `level` argument a default value of 0 and then load your code into the IDLE editor. Press F5 to load the code into the shell and then follow along to confirm that the latest version of your function works as expected. Start by defining a short list of lists and use the function to display the list on screen:

```
>>> names = ['John', 'Eric', ['Cleese', 'Idle'], 'Michael', ['Palin']]
>>> print_lol(names, 0)
```

```
John
Eric
  Cleese
  Idle
Michael
  Palin
```

The standard behavior works as expected, with nested lists indented.

Now try to do the same thing without specifying the second argument. Let's rely on the default value kicking in:

```
>>> print_lol(names)
```

```
John
Eric
  Cleese
  Idle
Michael
  Palin
```

Without specifying the second argument, the default is used and works, too.

Now specify a value for the second argument and note the change in the function's behavior:

```
>>> print_lol(names, 2)
```

```
    John
    Eric
      Cleese
      Idle
    Michael
      Palin
```

Specify an alternative value for the second argument and the indenting starts from that level.

One final example provides what looks like a silly value for the second argument. Look what happens:

```
>>> print_lol(names, -9)
```

```
John
Eric
Cleese
Idle
Michael
Palin
```

Using a negative value effectively switches OFF the indenting, as the count for "level" is unlikely to become a positive integer. This looks exactly like the original output from version 1.0.0, right?

Your module supports both APIs

Well done! It looks like your module is working well, as both APIs, the original 1.0.0 API *and* the newer 1.1.0 API, can now be used.

Let's take a moment to create and upload a new distribution for PyPI. As before, let's amend the `version` setting in the `setup.py` program:

```
name      = 'nester',
version   = '1.2.0',
py_modules = ['nester'],
```

Once again, be sure to change the value associated with "version" in "setup.py".

And with the code changes applied, upload this new version of your distribution to PyPI:

File Edit Window Help UploadThree

```
$ python3 setup.py sdist upload
running sdist
running check
reading manifest file 'MANIFEST'
creating nester-1.2.0
making hard links in nester-1.2.0...
hard linking nester.py -> nester-1.2.0
hard linking setup.py -> nester-1.2.0
Creating tar archive
removing 'nester-1.2.0' (and everything under it)
running upload
Submitting dist/nester-1.2.0.tar.gz to http://pypi.python.org/pypi
Server response (200): OK
$
```

This all looks fine and dandy.

Success! The messages from `setup.py` confirm that the your latest version of `nester` is up on PyPI. Let's hope this one satisfies all of your users.



Consider your code carefully. How might some of your users still have a problem with this version of your code?

Your API is still not right

Although the API lets your users invoke the function in its original form, *the nesting is switched on by default*. This behavior is not required by everyone and some people aren't at all happy.



Of course, if you have some functionality that really ought to be optional (that is, *not* the default), you should adjust your code to make it so. But how?

One solution is to **add a third argument** which is set to `True` when the indenting is required and `False` otherwise. If you ensure that this argument is `False` by default, the original functionality becomes the default behavior and users of your code have to request the new indenting feature explicitly.

Let's look at adding this final revision.



Exercise

- 1 Amend your module one last time to add a third argument to your function. Call your argument `indent` and set it initially to the value `False`—that is, do not switch on indentation by default. In the body of your function, use the value of `indent` to control your indentation code.

Note: to save a bit of space, the comments from the module are not shown here. Of course, you need to make the necessary adjustments to your comments to keep them in sync with your code.

Put the extra argument here.

```
def print_lol(the_list, _____, level=0):

    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, _____, level+1)
        else:
            _____
            for tab_stop in range(level):
                print("\t", end='')
            print(each_item)
```

What needs to go in here?

Add a line of code to control when indenting occurs.

- 2 With your new code additions in place, provide the edit you would recommend making to the `setup.py` program prior to uploading this latest version of your module to PyPI:

.....

.....

- 3 Provide the command you would use to upload your new distribution to PyPI:

.....

.....



Exercise Solution

- 1 You were to amend your module one last time to add a third argument to your function. You were to call your argument `indent` and set it initially to the value `False`—that is, do not switch on indentation by default. In the body of your function, you were to use the value of `indent` to control your indentation code.

```
def print_lol(the_list, indent=False, level=0):
    for each_item in the_list:
        if isinstance(each_item, list):
            print_lol(each_item, indent, level+1)
        else:
            if indent:
                for tab_stop in range(level):
                    print("\t", end='')
                print(each_item)
```

Did you include the default value?

Your signature has changed, so be sure to update this invocation.

A simple "if" statement does the trick.

Don't forget the colon at the end of the "if" line.

A sweet alternative to this "for" loop is this code: `print("\t" * level, end="")`.

- 2 With your new code additions in place, you were to provide the edit you would recommend making to the `setup.py` program prior to uploading this latest version of your module to PyPI:

.....
 Edit "setup.py" so that it reads: `version = '1.3.0',`

It's a new version of your module, so be sure to change the value associated with "version" in your "setup.py" file.

- 3 You were to provide the command you would use to upload your new distribution to PyPI:

.....
`python3 setup.py sdist upload`

Remember: if you are on Windows use `"C:\Python31\python.exe"` instead of `"python3"`.



An IDLE Session

A final test of the functionality should convince you that your module is now working exactly the way you and your users want it to. Let's start with the original, default behavior:

```
>>> names = ['John', 'Eric', ['Cleese', 'Idle'], 'Michael', ['Palin']]
```

```
>>> print_lol(names)
```

```
John
Eric
Cleese
Idle
Michael
Palin
```

The original, default functionality is restored (that should please Bob).

Next, turn on indentation by providing `True` as the second argument:

```
>>> names = ['John', 'Eric', ['Cleese', 'Idle'], 'Michael', ['Palin']]
```

```
>>> print_lol(names, True)
```

```
John
Eric
```

```
    Cleese
    Idle
```

```
Michael
    Palin
```

By providing a second argument, it's possible to switch on indented output (keeping Laura happy).

And, finally, control where indentation begins by providing a third argument value:

```
>>> names = ['John', 'Eric', ['Cleese', 'Idle'], 'Michael', ['Palin']]
```

```
>>> print_lol(names, True, 4)
```

```
John
Eric
```

```
    Cleese
    Idle
```

```
Michael
    Palin
```

Indenting from a specific tab-stop is also possible.

Do this!

Go ahead and edit your `setup.py` file; then upload your distribution to PyPI.

Your module's reputation is restored

Congratulations! Word of your new and improved module is spreading fast.



Your Python skills are starting to build

You've created a useful module, made it shareable, and uploaded it to the PyPI website. Programmers all over the world are downloading and using your code in their projects.

Keep up the good work.



Your Python Toolbox

You've got Chapter 2 under your belt and you've added some key Python goodies to your toolbox.

Python Lingo

- Use a "triple-quoted string" to include a multiple-line comment in your code.
- "PyPI" is the Python Package Index and is well worth a visit.
- A "namespace" is a place in Python's memory where names exist.
- Python's main namespace is known as `__main__`.

IDLE Notes

- Press F5 to "run" the code in the IDLE edit window.
- When you press F5 to "load" a module's code into the IDLE shell, the module's names are specifically imported into IDLE's namespace. This is a convenience when using IDLE. Within your code, you need to use the import statement explicitly.



BULLET POINTS

- A **module** is a text file that contains Python code.
- The **distribution utilities** let you turn your module into a shareable package.
- The `setup.py` program provides *metadata* about your module and is used to build, install, and upload your packaged distribution.
- Import your module into other programs using the **import** statement.
- Each module in Python provides its own **namespace**, and the namespace name is used to qualify the module's functions when invoking them using the `module.function()` form.
- Specifically import a function from a module into the current namespace using the `from module import function` form of the import statement.
- Use `#` to comment-out a line of code or to add a short, one-line comment to your program.
- The built-in functions (BIFs) have their own namespace called `__builtins__`, which is automatically included in every Python program.
- The `range()` BIF can be used with **for** to iterate a fixed number of times.
- Including `end=' '` as a argument to the `print()` BIF switches off its automatic inclusion of a new-line on output.
- Arguments to your functions are **optional** if you provide them with a **default value**.

